

**FIRAT ÜNİVERSİTESİ**  
**Mühendislik Fakültesi**  
**Bilgisayar Mühendisliği Bölümü**

**BMÜ329 VERİ TABANI SİSTEMLERİ**  
**DÖNEM PROJESİ RAPORU**

**Bankacılık ve Finans Yönetim Sistemi**

|                       |                           |
|-----------------------|---------------------------|
| <b>Grup No</b>        | 12                        |
| <b>Takım Üyesi 1</b>  | Şeyhmus Elik              |
| <b>Takım Üyesi 2</b>  | Mustafa Hayrullah Yıldız  |
| <b>Takım Üyesi 3</b>  | Emirhan Efe Gözpınar      |
| <b>Ders Sorumlusu</b> | Dr.Öğr. Üyesi Ertan Bütün |
| <b>Teslim Tarihi</b>  | [27/12/2025]              |

# İÇİNDEKİLER

## 1. GİRİŞ VE PROJE TANIMI

### 1.1. Projenin Amacı ve Kapsamı

### 1.2. Hedef Kullanıcılar

## 2. PROJE GEREKSİNİMLERİ

### 2.1. Fonksiyonel Gereksinimler

### 2.2. Fonksiyonel Olmayan Gereksinimler

### 2.3. İş Kuralları ve Kısıtlamalar

## 3. VERİ TABANI TASARIMI - VARLIK-İLİŞKİ (E-R) MODELİ

### 3.1. E-R Diyagramı

## 4. İLİŞKİSEL ŞEMALAR

### 4.1. E-R'dan İlişkisel Şemalarına Dönüşüm

## 5. NORMALİZASYON

### 5.1. Normalizasyon Süreci 5.2. Son Normalize Edilmiş Şema

## 6. SQL SERVER VERİ TABANI ŞEMASI

### 6.1. Tablo Oluşturma Komutları

### 6.2. Veri Tabanı Diyagramı

## 7. ÖRNEK VERİLER

### 7.1. Veri Ekleme Komutları (INSERT)

## 8. SQL KOMUTLARI VE SCRIPT DOSYALARI

### 8.1. Gereksinim Bazlı SQL Komutları ve Script Dosyaları

## 9. SAKLI YORDAM VE TETİKLEYİCİ

9.1. Saklı Yordamlar - 9.1.a. sp\_DovizIslemi - 9.1.b. sp\_KartBorcOdeme - 9.1.c. sp\_YatirimAlimi - 9.1.d. sp\_ParaTransferi - 9.1.e. sp\_HesapKapat 9.2. Tetikleyiciler (Triggers) - 9.2.a. trg\_KampanyaTarihKontrol - 9.2.b. trg\_ParaBirimiTarihGuncelle - 9.2.c. trg\_HareketDegisiklikEngelle - 9.2.d. trg\_BasvuruDurumLog - 9.2.e. trg\_HesapSilmeKorumasi

## 10. TRANSACTION YÖNETİMİ

10.1. Döviz Alım Satım Senaryosu 10.2. Kredi Kartı Borç Ödeme Senaryosu  
10.3. Hesaplar Arası Para Transferi Senaryosu 10.4. Yatırım Varlığı Alımı  
Senaryosu

## 11. TAKIM ÇALIŞMASI VE GÖREV DAĞILIMI

11.1. Takım Üyeleri ve Roller

11.2. Gerçekleştirilen İşler ve Sorumluluk Matrisi

## 12. SONUÇ VE DEĞERLENDİRME

12.1. Elde Edilen Deneyimler ve Kazanımlar

12.2. Karşılaşılan Zorluklar ve Çözümler 12.3. Genel Değerlendirme

# 1. GİRİŞ VE PROJE TANIMI

## 1.1 Projenin Amacı ve Kapsamı

Bu proje, modern bankacılığın temel gereksinimleri olan güvenlik, hız ve veri bütünlüğünü tek bir çatı altında toplayan kapsamlı bir "Çekirdek Bankacılık" altyapısıdır. Sistemin temel amacı, sadece para transferlerini yönetmek değil; müşterilerin vadesiz hesaplarından yatırım portföylerine, kredi kartı borç ödemelerinden döviz alım satımlarına kadar tüm finansal yaşam döngüsünü dijital ortamda güvenle sürdürebilmelerini sağlamaktır. Geliştirilen bu yapı, bankacılık işlemlerinde sıkça karşılaşılan "işlem sırasında paranın kaybolması" riskini, veritabanı seviyesinde kurgulanan "Ya Hep Ya Hiç" (Transaction) prensibiyle ortadan kaldırır; bir transfer sırasında gönderenden para düşülüp alıcıya geçmezse işlem tamamen iptal edilir ve bakiye korunur.

Projenin teknik kapsamı, finansal işlemlerin ötesinde, sıkı bir güvenlik ve denetim mekanizmasını da içerir. Sistem, geçmişe dönük hesap hareketlerinin silinmesini veya değiştirilmesini engelleyen özel tetikleyicilerle (triggers) donatılmıştır, bu sayede hem iç hem de dış kaynaklı veri manipülasyonlarının önüne geçilir. Ayrıca, müşteri başvurularından sistem hatalarına kadar her türlü kritik olay anlık olarak loglanarak tam bir izlenebilirlik sağlanır. Özetle bu proje, finansal çeşitliliği yüksek güvenlik standartlarıyla birleştirerek, hem banka operasyonlarını hatasız yönetmeyi hem de müşteri varlıklarını koruma altına almayı hedefler.

## 1.2 Hedef Kullanıcılar

Bu sistemin hitap ettiği kitle, hizmeti alanlar ve hizmeti sağlayanlar olarak iki ana grupta toplanır. Birinci grupta, günlük bankacılık işlemlerini, fatura ödemelerini ve para transferlerini hızlıca yapmak isteyen bireysel müşteriler ile birikimlerini döviz, hisse senedi veya fon gibi araçlarla değerlendirmek isteyen aktif yatırımcılar yer alır. Bu kullanıcılar için sistem, varlıklarını tek bir noktadan yönetebilecekleri şeffaf bir platform sunar.

İkinci grupta ise bankanın iç operasyonlarını yürüten profesyoneller bulunur. Şubelerde müşteri taleplerini ve kredi başvurularını inceleyen banka personeli, sistemin güvenliğini ve genel ayarlarını yöneten yöneticiler (Admin) ve olası şüpheli işlemleri denetleyen operasyon ekipleri bu altyapının diğer kullanıcılarıdır. Sistem, yetki bazlı erişim kontrolü sayesinde her kullanıcının sadece kendi rolüne uygun işlemleri yapmasına olanak tanıyarak kurumsal bir çalışma disiplini sağlar

## 2. PROJE GEREKSİNİMLERİ

### 2.1 Fonksiyonel Gereksinimler

| ID    | Gereksinim Adı              | Açıklama                                                                                                                                                                   |
|-------|-----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-01 | Kullanıcı Kaydı ve Giriş    | Kullanıcılar (Müşteri/Personel) sisteme kayıt olabilmeli, kimlik doğrulama ile giriş yapabilmeli ve rollerine uygun arayüze erişebilmelidir <sup>1</sup> .                 |
| FR-02 | Hesap Yönetimi              | Müşteriler farklı para birimlerinde (TRY, USD, EUR) vadesiz veya yatırım hesapları açabilmeli, kapatabilmeli ve bakiyelerini görüntüleyebilmelidir <sup>22</sup> .         |
| FR-03 | Para Transferi (EFT/Havale) | Müşteriler, kendi hesapları arasında veya başka müşterilerin hesaplarına, bakiye kontrolü yapılarak güvenli para transferi gerçekleştirebilmelidir <sup>3333</sup> .       |
| FR-04 | Döviz Alım-Satım            | Sistem, anlık kur bilgileri üzerinden müşterilerin hesapları arasında döviz bozdurma (satış) veya döviz alma işlemlerini gerçekleştirmesini sağlamalıdır <sup>4444</sup> . |
| FR-05 | Yatırım İşlemleri           | Müşteriler hisse senedi, fon veya tahvil gibi yatırım araçlarını vadesiz hesap bakiyelerini kullanarak alabilmeli ve portföylerini takip edebilmelidir <sup>5555</sup> .   |
| FR-06 | Kredi Kartı Borç Ödeme      | Müşteriler, tanımlı kredi kartlarının borçlarını vadesiz hesaplarından anlık olarak ödeyebilmeli ve işlem sonucu karta yansıtılmalıdır <sup>6666</sup> .                   |
| FR-07 | Kampanya Başvurusu          | Müşteriler aktif banka kampanyalarını inceleyip başvuruda bulunabilmeli; personel bu başvuruları onaylayıp reddedebilmelidir <sup>7</sup> .                                |
| FR-08 | Teknik Destek Talebi        | Müşteriler işlem hataları veya hesap sorunları için destek talebi oluşturabilmeli, personel bu talepleri yanıtlayıp çözümleyebilmelidir <sup>8</sup> .                     |

### 2.2 Fonksiyonel Olmayan Gereksinimler

| ID     | Gereksinim Adı            | Açıklama                                                                                                                                                                          |
|--------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Veri Güvenliği (Security) | Kullanıcı şifreleri, kredi kartı PIN ve CVV kodları veritabanında açık metin olarak değil, hashlenmiş (SHA-256 vb.) ve tuzlanmış (salted) şekilde saklanmalıdır <sup>9999</sup> . |

| ID     | Gereksinim Adı                  | Açıklama                                                                                                                                                                                                      |
|--------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-02 | İşlem Bütünlüğü (ACID)          | Para transferi ve döviz gibi finansal işlemler "Ya Hep Ya Hiç" (Transaction) prensibiyle çalışmalı; hata durumunda tüm işlem geri alınarak (Rollback) veri kaybı önlenmelidir <sup>101010101010101010</sup> . |
| NFR-03 | Denetim İzlenebilirliği (Audit) | Geçmiş hesap hareketlerinin silinmesi veya güncellenmesi engellenmeli; kritik veri değişiklikleri (örn. başvuru durumu) loglanarak kayıt altına alınmalıdır <sup>11111111</sup> .                             |
| NFR-04 | Veri Tutarlılığı                | Hesap bakiyesinin negatif olmaması, kampanya bitiş tarihinin başlangıçtan önce olmaması gibi iş kuralları veritabanı kısıtlamaları (Constraints/Triggers) ile zorlanmalıdır <sup>1212121212121212</sup> .     |
| NFR-05 | Veri Koruma                     | Banka hesapları fiziksel olarak silinememeli, sadece "Pasif" duruma getirilerek veri geçmişi korunmalıdır (Soft Delete) <sup>13131313</sup> .                                                                 |

## 2.2 İş Kuralları, Kısıtlamalar

### 2.2 İş Kuralları ve Kısıtlamalar

Banka Otomasyon Sistemi (BOS) projesinde veri bütünlüğünü, finansal güvenliği ve operasyonel tutarlılığı sağlamak amacıyla aşağıdaki iş kuralları ve kısıtlamalar tanımlanmıştır:

#### Genel Veri ve Kimlik Kuralları:

- **Benzersizlik (Uniqueness):** Her kullanıcının T.C. Kimlik/Kullanıcı Numarası (KullaniciNo) ve E-posta adresi sistem genelinde benzersiz olmalıdır. Aynı şekilde, her müşteriye atanan Müşteri Numarası (MusteriNo) ve Hesap Numarası (HesapNo) tekil olmalıdır.
- **Rol Kısıtlaması:** Sisteme kayıtlı kullanıcıların rolleri yalnızca 'Aday', 'Musteri', 'Personel' veya 'Admin' değerlerinden birini alabilir.
- **Şifreleme:** Kullanıcı giriş şifreleri, kredi kartı CVV kodları ve PIN numaraları veritabanında asla açık metin (plain-text) olarak saklanamaz; hashlenmiş formatta tutulmalıdır.

#### Finansal İşlem Kuralları:

- **Para Birimi Tutarlılığı:** `sp_ParaTransferi` prosedürü gereği, iki hesap arasında doğrudan para transferi yapılabilmesi için gönderen ve alıcı hesapların para birimleri (Örn: TRY -> TRY) aynı olmak zorundadır. Farklı para birimleri arasında doğrudan transfer engellenmiştir.
- **Bakiye Kontrolü:** Para transferi, döviz alımı veya yatırım işlemi öncesinde kaynak hesapta işlemin tutarını karşılayacak kadar bakiye bulunmalıdır. Bakiye yetersizse işlem veritabanı seviyesinde reddedilir (ROLLBACK).
- **Tutar Pozitifliği:** Transfer edilecek veya yatırılacak tutar her zaman 0'dan büyük olmalıdır (`CHECK (Tutar > 0)`). Hesap hareketlerinde ise tutar 0 olamaz (`CHECK (Tutar <> 0)`).

#### Güvenlik ve Denetim (Audit) Kuralları:

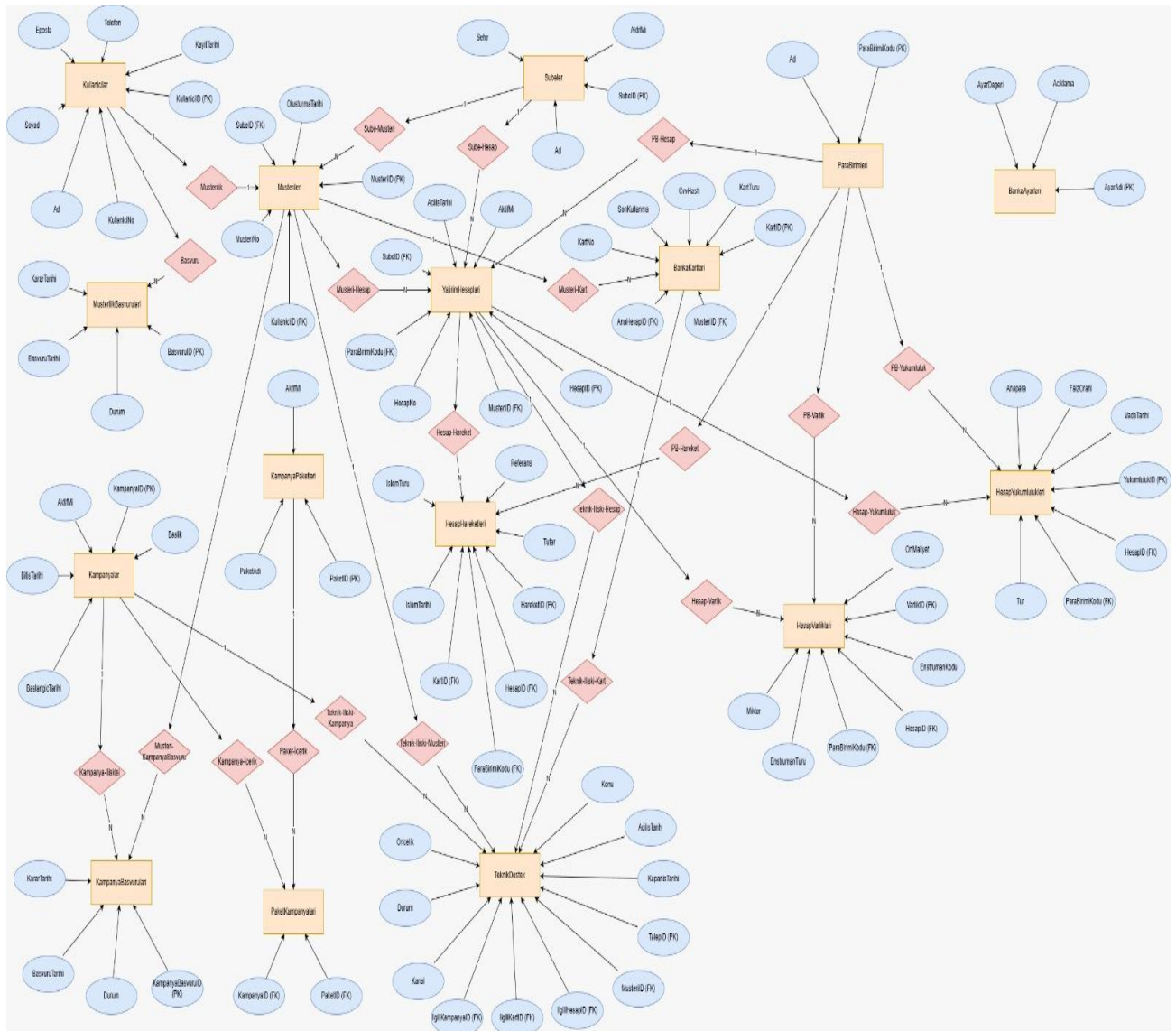
- **Değiştirilemezlik (Immutability):** `trg_HareketDegisiklikEngelle` tetikleyicisi gereği, geçmiş tarihli hesap hareketleri (para giriş-çıkışları) güvenlik ve denetim amacıyla silinemez veya güncellenemez.
- **Hesap Silme Koruması:** `trg_HesapSilmeKorumasi` tetikleyicisi gereği, içinde işlem geçmişi bulunan banka hesapları fiziksel olarak veritabanından silinemez; yalnızca "Pasif" (`AktifMi=0`) durumuna getirilebilir.
- **Başvuru Durumları:** Müşteri ve kampanya başvurularının durumu yalnızca 'Beklemede', 'Onaylandı' veya 'Reddedildi' değerlerini alabilir.

#### Operasyonel Kurallar:

- **Kampanya Tarihleri:** Bir kampanyanın `BitisTarihi`, `BaslangicTarihinden` önce olamaz. Bu kural trigger ile kontrol edilir.
- **Mükerrer Başvuru:** Bir müşteri, aynı kampanyaya birden fazla kez başvuruda bulunamaz (`UNIQUE (MusteriID, KampanyaID)`).
- **İşlem Türleri:** Hesap hareketleri tablosundaki işlem türleri sistem tarafından belirlenen standart değerler (Örn: 'Yatir', 'Cek', 'Alis', 'TransferGiden') dışında bir değer alamaz.

### 3. VERİ TABANI TASARIMI - VARLIK-İLİŞKİ (E-R) MODELİ

### 3.1 E-R Diyagramı





## 4. İLİŞKİSEL ŞEMALAR

### 4.1 E-R'dan İlişkisel Şemalarına Dönüşüm

1) **BankaAyarlari** (AyarAdi, AyarDegeri, Aciklama)

2) **Subeler** (SubeID, SubeAdi, Sehir, AktifMi)

3) **Kullaniciilar** (KullaniciID, KullaniciNo(U), Ad, Soyad, Eposta(U), Telefon, SifreHash, Tuz, Rol, KayitTarihi)

4) **Personel** (PersonelID, KullaniciID(FK)(U), SubeID(FK), Unvan)

5) **Musteriler** (MusteriID, KullaniciID(FK)(U), MusteriNo(U), SubeID(FK), Adres, Ilce, Sehir, OlusturmaTarihi)

6) **MusterilikBasvurulari** (BasvuruID, KullaniciID(FK), Durum, BasvuruTarihi, KararTarihi, OnaylayanPersonelID(FK))

7) **ParaBirimleri** (ParaBirimiKodu, Ad, AlisKuru, Satiskuru, GuncellemeTarihi)

8) **YatirimHesaplari** (HesapID, MusteriID(FK), ParaBirimiKodu(FK), SubeID(FK), HesapNo, AcilisTarihi, AktifMi)

**Kısıtlama:** UNIQUE(MusteriID, HesapNo)

9) **BankaKartlari** (KartID, MusteriID(FK), AnaHesapID(FK), KartNo(U), SonKullanma, CvvHash, PinHash, KartTuru)

10) **Transferler** (TransferID, GonderenHesapID(FK), AliciHesapID(FK), Tutar, ParaBirimiKodu(FK), IslemTarihi, Aciklama)

11) **HesapHareketleri** (HareketID, HesapID(FK), ParaBirimiKodu(FK), KartID(FK), TransferID(FK), IslemTarihi, IslemTuru, Tutar, Referans, IslemSonrasiBakiye)

12) **HesapVarliklari** (VarlikID, HesapID(FK), ParaBirimiKodu(FK), EnstrumanTuru, EnstrumanKodu, Miktar, OrtMaliyet)

13) **HesapYukumluluklari** (YukumlulukID, HesapID(FK), ParaBirimiKodu(FK), Tur, Anapara, FaizOrani, VadeTarihi)

14) **Kampanyalar** (KampanyaID, Baslik, Aciklama, BaslangicTarihi, BitisTarihi, AktifMi)

15) **KampanyaBasvurulari** (KampanyaBasvuruID, MusteriID(FK), KampanyaID(FK), Durum, BasvuruTarihi, KararTarihi, OnaylayanPersonelID(FK))

**Kısıtlama:** UNIQUE(MusteriID, KampanyaID)

**16) TeknikDestek** (TalepID, MusteriID(FK), IlgiliHesapID(FK), IlgiliKartID(FK), IlgiliKampanyaID(FK), Kanal, Konu, Aciklama, Oncelik, Durum, AcilisTarihi, KapanisTarihi)

**17) SistemLoglari** (LogID, TabloAdi, IslemTuru, IslemTarihi, Aciklama, Kullanici)

## 5. NORMALİZASYON

### 5.1 Normalizasyon Süreci

#### Adım 1: 1. Normal Form (1NF) - Atomiklik

- **Kural:** Her sütun tek bir değer tutmalı (Atomik olmalı), tekrar eden gruplar olmamalıdır.
- **Uygulama:**
  - Ham veri tasarımında Musteriler tablosunda "Adres" sütunu tek bir uzun metin olarak tutulabilirdi. Ancak tasarımda bu veri Adres, İlçe, Şehir olarak parçalanarak atomik hale getirilmiştir.
  - Kullanıcılar tablosunda "Ad Soyad" tek sütun yerine Ad ve Soyad olarak ayrılarak sorgulanabilirlik artırılmıştır.
  - **Sonuç:** Tabloların tamamı 1NF uyumludur.

#### Adım 2: 2. Normal Form (2NF) - Kısmi Bağımlılığın Kaldırılması

- **Kural:** Tablo 1NF olmalı ve bileşik anahtara (Composite Key) sahip tablolarda, anahtar olmayan sütunlar anahtarın *tamamına* bağlı olmalıdır.
- **Uygulama:**
  - HesapVarliklari tablosunu ele alalım. Eğer Primary Key (HesapID, EnstrumanKodu) olsaydı, Miktar ve OrtMaliyet her ikisine de bağlı olacaktı. Ancak tasarımda VarlikID (Identity) verilerek yapay bir anahtar (Surrogate Key) oluşturulmuş ve kısmi bağımlılık riski tasarım seviyesinde kaldırılmıştır.
  - KampanyaBasvurulari tablosunda MusteriID ve KampanyaID birleşimi yerine KampanyaBasvuruID kullanılarak her satırın tekil bir kimliği olması sağlanmıştır.
  - **Sonuç:** Tüm tablolar tekil Primary Key (Identity) kullandığı için 2NF uyumludur.

#### Adım 3: 3. Normal Form (3NF) - Geçişli Bağımlılığın Kaldırılması

- **Kural:** Tablo 2NF olmalı ve anahtar olmayan bir sütun, başka bir anahtar olmayan sütuna bağlı olmamalıdır (A -> B -> C ilişkisi olmamalı).
- **Uygulama:**
  - **Senaryo:** Eğer Musteriler tablosunda SubeAdi ve SubeŞehir alanları bulunsaydı; Müşteri -> ŞubeID -> ŞubeAdi şeklinde bir geçişli bağımlılık oluşacaktı.
  - **Çözüm:** Şube bilgileri Subeler adında ayrı bir tabloya taşınmış ve Musteriler tablosunda sadece SubeID (Foreign Key) bırakılmıştır.
  - **Senaryo:** Personel tablosunda şube bilgileri tekrar edilmemiş, sadece SubeID referansı kullanılmıştır.
  - **Sonuç:** Veri tekrarı önlenmiş, şube adı değiştiğinde binlerce müşteri kaydını güncelleme gerekliliği ortadan kalkmıştır.

#### Adım 4: BCNF (Boyce-Codd Normal Form) Analizi

- **Kural:** Her belirleyici (determinant) bir aday anahtar (candidate key) olmalıdır.
- **Analiz:**
  - Kullanıcılar tablosunda KullanıcıID PK'dir. Ancak KullanıcıNo (TCKN vb.) da UNIQUE tanımlanmıştır. Eposta da UNIQUE'tir. Herhangi bir sütun Eposta'ya bağlıysa, Eposta aynı zamanda bir aday anahtar olduğu için BCNF kuralı ihlal edilmez.
  - Musteriler tablosunda MusteriID PK'dir, ancak MusteriNo da UNIQUE'tir.
  - **İstisna/Kabul:** Musteriler tablosundaki İlçe -> Şehir bağımlılığı pratikte sıkça göz ardı edilir (Posta Kodu/İlçe veritabanı olmadığı sürece). Bu tasarımda adres metin bazlı tutulduğu için bu yapı 3NF/BCNF kabul edilir.

#### 5.2 Son Normalize Edilmiş Şema

Projenin Veritabanı Tasarımı (Bölüm 3) aşamasında, veritabanı performansını ve veri bütünlüğünü sağlamak amacıyla doğrudan 3. Normal Form (3NF) prensiplerine sadık kalınarak bir modelleme yapılmıştır. Bu nedenle, Bölüm 4'te sunulan ilişkisel şemalar halihazırda normalize edilmiş durumdadır. Yukarıdaki maddelerde, mevcut tasarımın normalizasyon kurallarına nasıl uyum sağladığı analiz edilerek ispatlanmıştır.

- **BankaAyarlari** (AyarAdi, AyarDegeri, Aciklama)
- **Subeler** (SubeID, SubeAdi, Sehir, AktifMi)
- **Kullanıcılar** (KullanıcıID, KullanıcıNo(U), Ad, Soyad, Eposta(U), Telefon, SifreHash, Tuz, Rol, KayitTarihi)
- **Personel** (PersonelID, KullanıcıID(FK)(U), SubeID(FK), Unvan)
- **Musteriler** (MusteriID, KullanıcıID(FK)(U), MusteriNo(U), SubeID(FK), Adres, İlçe, Sehir, OlusturmaTarihi)
- **MusterilikBasvurulari** (BasvuruID, KullanıcıID(FK), Durum, BasvuruTarihi, KararTarihi, OnaylayanPersonelID(FK))
- **ParaBirimleri** ParaBirimiKodu, Ad, AlisKuru, SatisKuru, GuncellemeTarihi)
- **YatirimHesaplari** (HesapID, MusteriID(FK), ParaBirimiKodu(FK), SubeID(FK), HesapNo, AcilisTarihi, AktifMi)
  - *Kısıtlama: UNIQUE(MusteriID, HesapNo)*
- **BankaKartlari** (KartID, MusteriID(FK), AnaHesapID(FK), KartNo(U), SonKullanma, CvvHash, PinHash, KartTuru)

- **Transferler** (TransferID, GonderenHesapID(FK), AliciHesapID(FK), Tutar, ParaBirimiKodu, IslemTarihi, Aciklama)
- **HesapHareketleri** (HareketID, HesapID\*, ParaBirimiKodu\*, KartID\*, TransferID\*, IslemTarihi, IslemTuru, Tutar, Referans, IslemSonrasiBakiye) <sup>20</sup>
- **HesapVarliklari** (VarlikID, HesapID\*, ParaBirimiKodu\*, EnstrumanTuru, EnstrumanKodu, Miktar, OrtMaliyet) <sup>21</sup>
- **HesapYukumlulukleri** (YukumlulukID, HesapID\*, ParaBirimiKodu, Tur, Anapara, FaizOrani, VadeTarihi) <sup>22</sup>
- **Kampanyalar** (KampanyaID, Baslik, Aciklama, BaslangicTarihi, BitisTarihi, AktifMi) <sup>23</sup>
- **KampanyaBasvurulari** (KampanyaBasvuruID, MusteriID\*, KampanyaID\*, Durum, BasvuruTarihi, KararTarihi, OnaylayanPersonelID\*) <sup>24</sup>
  - *Kısıtlama: UNIQUE(MusteriID, KampanyaID)*
- **TeknikDestek** (TalepID, MusteriID\*, IlgiliHesapID\*, IlgiliKartID\*, IlgiliKampanyaID\*, Kanal, Konu, Aciklama, Oncelik, Durum, AcilisTarihi, KapanisTarihi)

## 6. SQL SERVER VERİ TABANI ŞEMASI

### 6.1 Tablo Oluşturma Komutları

-- 1. Banka Ayarları (Sistem genelindeki sabitler)

```
CREATE TABLE BankaAyarlari (
  AyarAdi NVARCHAR(50) PRIMARY KEY,
  AyarDegeri NVARCHAR(255) NOT NULL,
  Aciklama NVARCHAR(MAX)
);
```

-- 2. Şubeler

```
CREATE TABLE Subeler (
  SubeID INT PRIMARY KEY IDENTITY(1,1),
  SubeAdi NVARCHAR(100) NOT NULL,
  Sehir NVARCHAR(50) NOT NULL,
  AktifMi BIT DEFAULT 1 -- 1: Aktif, 0: Pasif
);
```

-- 3. Kullanıcılar (GÜNCELLENDİ: Şifre ve Rol Eklendi)

```
CREATE TABLE Kullanicilar (
  KullaniciID INT PRIMARY KEY IDENTITY(1,1),
  KullaniciNo CHAR(11) NOT NULL UNIQUE, -- '10' ile başlayan no
  Ad NVARCHAR(50) NOT NULL,
  Soyad NVARCHAR(50) NOT NULL,
  Eposta NVARCHAR(100) NOT NULL UNIQUE,
  Telefon NVARCHAR(15),

  -- GÜVENLİK İÇİN EKLENEN ALANLAR
  SifreHash NVARCHAR(256) NOT NULL, -- Şifreler asla açık tutulmaz
  Tuz NVARCHAR(50) NOT NULL, -- Password Salt
  Rol NVARCHAR(20) NOT NULL CHECK (Rol IN ('Aday', 'Musteri', 'Personel', 'Admin')),

  KayitTarihi DATETIME DEFAULT GETDATE()
);
```

-- 4. Personel (YENİ TABLO: Çalışanları müşteriden ayırır)

```
CREATE TABLE Personel (
  PersonelID INT PRIMARY KEY IDENTITY(1,1),
  KullaniciID INT NOT NULL UNIQUE, -- Personel de bir kullanıcıdır
  SubeID INT, -- Personel hangi şubede çalışıyor?
  Unvan NVARCHAR(50),
  FOREIGN KEY (KullaniciID) REFERENCES Kullanicilar(KullaniciID),
  FOREIGN KEY (SubeID) REFERENCES Subeler(SubeID)
);
```

-- 5. Müşteriler (GÜNCELLENDİ: Adres Bilgisi Eklendi)

```
CREATE TABLE Musteriler (
  MusteriID INT PRIMARY KEY IDENTITY(1,1),
  KullaniciID INT NOT NULL UNIQUE,
  MusteriNo CHAR(10) NOT NULL UNIQUE, -- BankaKodu + 8 hane
  SubeID INT, -- Müşterinin bağlı olduğu şube

  -- EKLENEN ADRES BİLGİLERİ
  Adres NVARCHAR(250),
  Ilce NVARCHAR(50),
  Sehir NVARCHAR(50),
);
```

```

OlusturmaTarihi DATETIME DEFAULT GETDATE(),
FOREIGN KEY (KullaniciID) REFERENCES Kullanicilar(KullaniciID),
FOREIGN KEY (SubeID) REFERENCES Subeler(SubeID)
);

```

-- 6. Müşterilik Başvuruları (GÜNCELLENDİ: Onaylayan Kişi Eklendi)

```

CREATE TABLE MusterilikBasvurulari (
    BasvuruID INT PRIMARY KEY IDENTITY(1,1),
    KullaniciID INT NOT NULL,
    Durum NVARCHAR(20) DEFAULT 'Beklemede' CHECK (Durum IN ('Beklemede', 'Onaylandi', 'Reddedildi')),
    BasvuruTarihi DATETIME DEFAULT GETDATE(),
    KararTarihi DATETIME,

```

-- DENETİM İÇİN EKLENEN ALAN

OnaylayanPersonelID INT,

```

FOREIGN KEY (KullaniciID) REFERENCES Kullanicilar(KullaniciID),
FOREIGN KEY (OnaylayanPersonelID) REFERENCES Personel(PersonelID)
);

```

-- 7. Para Birimleri (GÜNCELLENDİ: Kur Bilgisi Eklendi)

```

CREATE TABLE ParaBirimleri (
    ParaBirimiKodu CHAR(3) PRIMARY KEY, -- TRY, USD, EUR
    Ad NVARCHAR(50) NOT NULL,
    AlisKuru DECIMAL(18, 4), -- Döviz işlemleri için gerekli
    SatisKuru DECIMAL(18, 4),
    GuncellemeTarihi DATETIME
);

```

-- 8. Yatırım Hesapları

```

CREATE TABLE YatirimHesaplari (
    HesapID INT PRIMARY KEY IDENTITY(1,1),
    MusteriID INT NOT NULL,
    ParaBirimiKodu CHAR(3) NOT NULL,
    SubeID INT,
    HesapNo NVARCHAR(20) NOT NULL, -- Müşteri bazında benzersiz olması gerekir
    AcilisTarihi DATETIME DEFAULT GETDATE(),
    AktifMi BIT DEFAULT 1,

    FOREIGN KEY (MusteriID) REFERENCES Musteriler(MusteriID),
    FOREIGN KEY (ParaBirimiKodu) REFERENCES ParaBirimleri(ParaBirimiKodu),
    FOREIGN KEY (SubeID) REFERENCES Subeler(SubeID),
    UNIQUE (MusteriID, HesapNo) -- Bir müşterinin aynı no'lu iki hesabı olamaz
);

```

-- 9. Banka Kartları (GÜNCELLENDİ: PIN Hash Eklendi)

```

CREATE TABLE BankaKartlari (
    KartID INT PRIMARY KEY IDENTITY(1,1),
    MusteriID INT NOT NULL,
    AnaHesapID INT, -- Debit kart ise bağlı olduğu hesap
    KartNo CHAR(16) NOT NULL UNIQUE,
    SonKullanma CHAR(5) NOT NULL, -- MM/YY formatı
    CvvHash NVARCHAR(256) NOT NULL, -- Güvenlik kodu hash'i
    PinHash NVARCHAR(256) NOT NULL, -- EKLENEN: ATM şifresi hash'i
    KartTuru NVARCHAR(10) CHECK (KartTuru IN ('Debit', 'Credit')),

    FOREIGN KEY (MusteriID) REFERENCES Musteriler(MusteriID),
    FOREIGN KEY (AnaHesapID) REFERENCES YatirimHesaplari(HesapID)
);

```

-- 10. Transferler (YENİ TABLO: İki hesap arası para transferini yönetir)

```
CREATE TABLE Transferler (
  TransferID INT PRIMARY KEY IDENTITY(1,1),
  GonderenHesapID INT NOT NULL,
  AliciHesapID INT NOT NULL,
  Tutar DECIMAL(18, 2) NOT NULL CHECK (Tutar > 0),
  ParaBirimiKodu CHAR(3) NOT NULL,
  IslemTarihi DATETIME DEFAULT GETDATE(),
  Aciklama NVARCHAR(100),

  FOREIGN KEY (GonderenHesapID) REFERENCES YatirimHesaplari(HesapID),
  FOREIGN KEY (AliciHesapID) REFERENCES YatirimHesaplari(HesapID)
  -- Not: Bu tabloya kayıt girilince Trigger ile HesapHareketleri tablosuna
  -- otomatik olarak bir "Giden" ve bir "Gelen" kaydı atılması önerilir.
);
```

-- 11. Hesap Hareketleri (GÜNCELLENDİ: Bakiye ve Transfer Ref Eklendi)

```
CREATE TABLE HesapHareketleri (
  HareketID INT PRIMARY KEY IDENTITY(1,1),
  HesapID INT NOT NULL,
  ParaBirimiKodu CHAR(3) NOT NULL,
  KartID INT, -- Kartla yapıldıysa
  TransferID INT, -- EKLENEN: Transfer işlemiyse referansı

  IslemTarihi DATETIME DEFAULT GETDATE(),
  IslemTuru NVARCHAR(20) CHECK (IslemTuru IN ('Yatir', 'Cek', 'Alis', 'Satis', 'Ucret', 'Faiz', 'TransferGiden',
  'TransferGelen')),
  Tutar DECIMAL(18, 2) NOT NULL CHECK (Tutar <> 0),
  Referans NVARCHAR(50),

  IslemSonrasiBakiye DECIMAL(18, 2), -- EKLENEN: Performans için anlık bakiye

  FOREIGN KEY (HesapID) REFERENCES YatirimHesaplari(HesapID),
  FOREIGN KEY (ParaBirimiKodu) REFERENCES ParaBirimleri(ParaBirimiKodu),
  FOREIGN KEY (KartID) REFERENCES BankaKartlari(KartID),
  FOREIGN KEY (TransferID) REFERENCES Transferler(TransferID)
);
```

-- 12. Hesap Varlıkları (Hisse, Fon, Döviz gibi enstrümanlar)

```
CREATE TABLE HesapVarliklari (
  VarlikID INT PRIMARY KEY IDENTITY(1,1),
  HesapID INT NOT NULL,
  ParaBirimiKodu CHAR(3) NOT NULL, -- Varlığın değerlendirildiği birim
  EnstrumanTuru NVARCHAR(20) CHECK (EnstrumanTuru IN ('Hisse', 'Fon', 'Tahvil', 'Doviz')),
  EnstrumanKodu NVARCHAR(20) NOT NULL, -- Örn: THYAO, USD
  Miktar DECIMAL(18, 4) DEFAULT 0,
  OrtMaliyet DECIMAL(18, 2) DEFAULT 0,

  FOREIGN KEY (HesapID) REFERENCES YatirimHesaplari(HesapID),
  FOREIGN KEY (ParaBirimiKodu) REFERENCES ParaBirimleri(ParaBirimiKodu)
);
```

-- 13. Hesap Yükümlülükleri (Krediler vb.)

```
CREATE TABLE HesapYukumluluklari (
  YukumlulukID INT PRIMARY KEY IDENTITY(1,1),
  HesapID INT NOT NULL,
  ParaBirimiKodu CHAR(3) NOT NULL,
  Tur NVARCHAR(20) CHECK (Tur IN ('Kredi', 'Marjin')),
);
```



Anapara DECIMAL(18, 2) DEFAULT 0,  
 FaizOrani DECIMAL(5, 2) DEFAULT 0,  
 VadeTarihi DATETIME,

FOREIGN KEY (HesapID) REFERENCES YatirimHesaplari(HesapID)  
 );

#### -- 14. Kampanyalar

CREATE TABLE Kampanyalar (  
 KampanyaID INT PRIMARY KEY IDENTITY(1,1),  
 Baslik NVARCHAR(100) NOT NULL,  
 Aciklama NVARCHAR(MAX),  
 BaslangicTarihi DATETIME NOT NULL,  
 BitisTarihi DATETIME,  
 AktifMi BIT DEFAULT 1  
 );

#### -- 15. Kampanya Başvuruları

CREATE TABLE KampanyaBasvurulari (  
 KampanyaBasvuruID INT PRIMARY KEY IDENTITY(1,1),  
 MusteriID INT NOT NULL,  
 KampanyaID INT NOT NULL,  
 Durum NVARCHAR(20) DEFAULT 'Beklemede',  
 BasvuruTarihi DATETIME DEFAULT GETDATE(),  
 KararTarihi DATETIME,  
 OnaylayanPersonelID INT, -- Onaylayan personel  
  
 FOREIGN KEY (MusteriID) REFERENCES Musteriler(MusteriID),  
 FOREIGN KEY (KampanyaID) REFERENCES Kampanyalar(KampanyaID),  
 FOREIGN KEY (OnaylayanPersonelID) REFERENCES Personel(PersonelID),  
 UNIQUE (MusteriID, KampanyaID) -- Aynı kampanyaya mükerrer başvuru engellendi  
 );

#### -- 16. Teknik Destek

CREATE TABLE TeknikDestek (  
 TalepID INT PRIMARY KEY IDENTITY(1,1),  
 MusteriID INT NOT NULL,  
 -- İlişkili varlıklar (Hepsine NULL verilebilir)  
 IlgiliHesapID INT,  
 IlgiliKartID INT,  
 IlgiliKampanyaID INT,  
  
 Kanal NVARCHAR(20) CHECK (Kanal IN ('Telefon', 'E-Posta', 'Uygulama')),  
 Konu NVARCHAR(100) NOT NULL,  
 Aciklama NVARCHAR(MAX) NOT NULL,  
 Oncelik NVARCHAR(10) CHECK (Oncelik IN ('Dusuk', 'Orta', 'Yuksek')),  
 Durum NVARCHAR(20) DEFAULT 'Acik' CHECK (Durum IN ('Acik', 'Cozumde', 'Kapali')),  
  
 AcilisTarihi DATETIME DEFAULT GETDATE(),  
 KapanisTarihi DATETIME,  
  
 FOREIGN KEY (MusteriID) REFERENCES Musteriler(MusteriID),  
 FOREIGN KEY (IlgiliHesapID) REFERENCES YatirimHesaplari(HesapID),  
 FOREIGN KEY (IlgiliKartID) REFERENCES BankaKartlari(KartID),  
 FOREIGN KEY (IlgiliKampanyaID) REFERENCES Kampanyalar(KampanyaID)  
 );

#### -- Triggerların log atacağı tablo

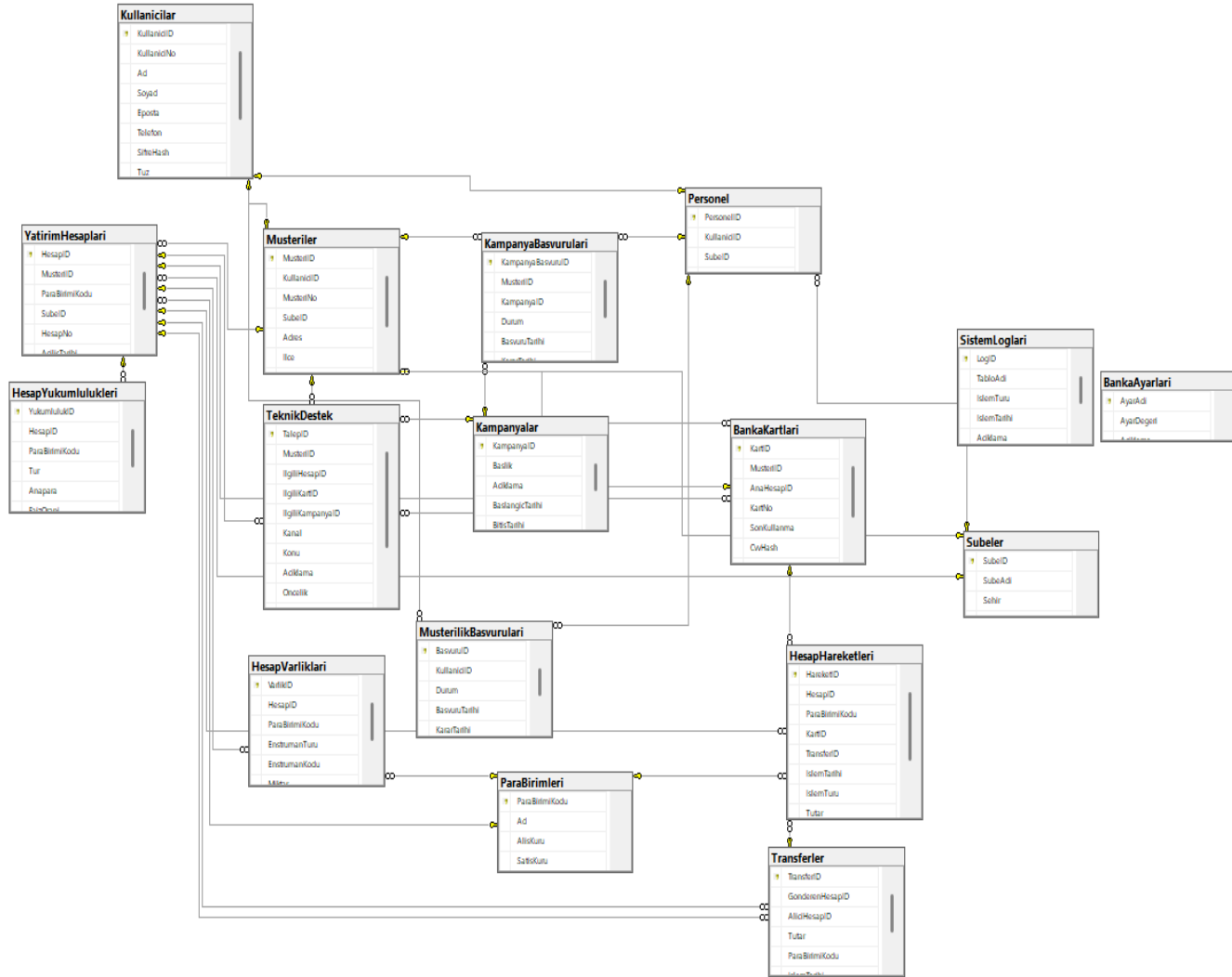
CREATE TABLE SistemLoglari (  
 );

```

LogID INT PRIMARY KEY IDENTITY(1,1),
TabloAdi NVARCHAR(50),
IslemTuru NVARCHAR(20), -- INSERT, UPDATE, DELETE
IslemTarihi DATETIME DEFAULT GETDATE(),
Aciklama NVARCHAR(MAX),
Kullanici NVARCHAR(100) DEFAULT SYSTEM_USER -- İşlemi yapan veritabanı kullanıcısı
);

```

### 6.3 Veri Tabanı Diyagramı



## 7. ÖRNEK VERİLER

### 7.1 Veri Ekleme Komutları (INSERT)

-- 1. Banka Ayarları

```
INSERT INTO BankaAyarlari (AyarAdi, AyarDegeri, Aciklama) VALUES
('EFT_Limiti', '50000', 'Günlük maksimum EFT limiti'),
('Varsayılan_Faiz', '3.50', 'Varsayılan mevduat faiz oranı');
```

-- 2. Şubeler

```
INSERT INTO Subeler (SubeAdi, Sehir, AktifMi) VALUES
('Merkez Şube', 'İstanbul', 1),
('Kadıköy Şubesi', 'İstanbul', 1),
('Kızılay Şubesi', 'Ankara', 1);
```

-- 3. Para Birimleri (Kur bilgileriyle)

```
INSERT INTO ParaBirimleri (ParaBirimiKodu, Ad, AlisKuru, SatisKuru, GuncellemeTarihi) VALUES
('TRY', 'Türk Lirası', 1.0000, 1.0000, GETDATE()),
('USD', 'Amerikan Doları', 32.5000, 32.8500, GETDATE()),
('EUR', 'Euro', 35.1000, 35.6000, GETDATE());
```

-- 4. Kampanyalar (Tarih kontrolü trigger'ını test etmek için geçerli tarihler)

```
INSERT INTO Kampanyalar (Baslik, Aciklama, BaslangicTarihi, BitisTarihi, AktifMi) VALUES
('Yeni Müşteri Kredisi', '0 faizli hoşgeldin kredisi', '2024-01-01', '2025-12-31', 1),
('Yaz Yatırım Fırsatı', 'Döviz işlemlerinde düşük makas', '2024-06-01', '2024-09-01', 1);
```

-- 5. Kullanıcılar (Şifreler hashlenmiş varsayılır)

-- Admin Kullanıcısı

```
INSERT INTO Kullanicilar (KullaniciNo, Ad, Soyad, Eposta, Telefon, SifreHash, Tuz, Rol) VALUES
('10000000001', 'Sistem', 'Admin', 'admin@banka.com', '5551112233', 'hash_admin_123', 'salt1', 'Admin');
```

-- Personel Kullanıcısı

```
INSERT INTO Kullanicilar (KullaniciNo, Ad, Soyad, Eposta, Telefon, SifreHash, Tuz, Rol) VALUES
('10000000002', 'Ahmet', 'Yılmaz', 'ahmet.yilmaz@banka.com', '5552223344', 'hash_personel_1', 'salt2', 'Personel');
```

-- Müşteri 1 (Zengin Müşteri)

```
INSERT INTO Kullanicilar (KullaniciNo, Ad, Soyad, Eposta, Telefon, SifreHash, Tuz, Rol) VALUES
('20000000001', 'Ayşe', 'Demir', 'ayse.demir@gmail.com', '5553334455', 'hash_musteri_1', 'salt3', 'Musteri');
```

-- Müşteri 2 (Standart Müşteri)

```
INSERT INTO Kullanicilar (KullaniciNo, Ad, Soyad, Eposta, Telefon, SifreHash, Tuz, Rol) VALUES
('20000000002', 'Mehmet', 'Öztürk', 'mehmet.ozturk@hotmail.com', '5554445566', 'hash_musteri_2', 'salt4', 'Musteri');
```

-- Aday Kullanıcı (Başvuru Testi İçin)

```
INSERT INTO Kullanicilar (KullaniciNo, Ad, Soyad, Eposta, Telefon, SifreHash, Tuz, Rol) VALUES
('30000000001', 'Veli', 'Bak', 'veli.bak@yahoo.com', '5559998877', 'hash_aday_1', 'salt5', 'Aday');
```

-- 6. Personel Detayları

-- Ahmet Yılmaz'ı Kadıköy şubesine bağlıyoruz.

```
INSERT INTO Personel (KullaniciID, SubeID, Unvan) VALUES
(2, 2, 'Gişe Görevlisi'); -- KullaniciID 2 (Ahmet)
```

-- 7. Müşteri Detayları

-- Ayşe Demir (Merkez Şube)

```
INSERT INTO Musteriler (KullaniciID, MusteriNo, SubeID, Adres, Ilce, Sehir) VALUES
(3, 'MUST000001', 1, 'Bağdat Caddesi No:5', 'Kadıköy', 'İstanbul');
```

-- Mehmet Öztürk (Kızılay Şube)

```
INSERT INTO Musteriler (KullaniciID, MusteriNo, SubeID, Adres, Ilce, Sehir) VALUES
(4, 'MUST000002', 3, 'Karanfil Sokak No:12', 'Çankaya', 'Ankara');
```

```

-- 8. Yatırım Hesapları
-- Ayşe Demir için TRY Hesabı
INSERT INTO YatirimHesapları (MusteriID, ParaBirimiKodu, SubeID, HesapNo) VALUES
(1, 'TRY', 1, 'TR-1001-001'); -- MusteriID 1 (Ayşe)

-- Ayşe Demir için USD Hesabı
INSERT INTO YatirimHesapları (MusteriID, ParaBirimiKodu, SubeID, HesapNo) VALUES
(1, 'USD', 1, 'TR-1001-002');

-- Mehmet Öztürk için TRY Hesabı
INSERT INTO YatirimHesapları (MusteriID, ParaBirimiKodu, SubeID, HesapNo) VALUES
(2, 'TRY', 3, 'TR-2002-001'); -- MusteriID 2 (Mehmet)

-- 9. Banka Kartları
-- Ayşe'ye Kredi Kartı
INSERT INTO BankaKartları (MusteriID, AnaHesapID, KartNo, SonKullanma, CvvHash, PinHash, KartTuru)
VALUES
(1, NULL, '4545123456789012', '12/28', 'hash_cvv_1', 'hash_pin_1', 'Credit');

-- Mehmet'e Debit Kart (TRY hesabına bağlı)
INSERT INTO BankaKartları (MusteriID, AnaHesapID, KartNo, SonKullanma, CvvHash, PinHash, KartTuru)
VALUES
(2, 3, '5555987654321098', '06/27', 'hash_cvv_2', 'hash_pin_2', 'Debit'); -- AnaHesapID 3 (Mehmet'in TRY hesabı)

-- 10. Başlangıç Bakiyeleri (Hesap Hareketleri)
-- Ayşe hesabına 50.000 TL yatırıyor
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, Referans, IslemSonrasiBakiye)
VALUES
(1, 'TRY', 'Yatir', 50000.00, 'ATM Para Yatırma', 50000.00);

-- Ayşe 1000 USD alıyor (Kur: 32.85 -> 32.850 TL gider)
-- Hareket 1: TL Çıkışı
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, Referans, IslemSonrasiBakiye)
VALUES
(1, 'TRY', 'Alis', -32850.00, 'Döviz Alışı', 17150.00);
-- Hareket 2: USD Girişi
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, Referans, IslemSonrasiBakiye)
VALUES
(2, 'USD', 'Alis', 1000.00, 'Döviz Alışı', 1000.00);

-- 11. Para Transferi (Ayşe'den Mehmet'e Havale)
-- Önce Transfer kaydı (Parent)
INSERT INTO Transferler (GonderenHesapID, AlıcıHesapID, Tutar, ParaBirimiKodu, Aciklama) VALUES
(1, 3, 5000.00, 'TRY', 'Kira Ödemesi');

-- Transferin Hareket Kayıtları
DECLARE @TransferID INT = SCOPE_IDENTITY(); -- Eklenen son ID'yi al

-- Gönderen (Ayşe)
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, TransferID, IslemTuru, Tutar, Referans,
IslemSonrasiBakiye) VALUES
(1, 'TRY', @TransferID, 'TransferGiden', -5000.00, 'Kira Ödemesi', 12150.00);

-- Alıcı (Mehmet)
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, TransferID, IslemTuru, Tutar, Referans,
IslemSonrasiBakiye) VALUES
(3, 'TRY', @TransferID, 'TransferGelen', 5000.00, 'Kira Ödemesi', 5000.00);

-- 12. Hesap Varlıkları (Yatırım)
-- Ayşe 100 Lot THYAO alıyor (Maliyeti 300 TL varsayalım)

```

```
INSERT INTO HesapVarliklari (HesapID, ParaBirimiKodu, EnstrumanTuru, EnstrumanKodu, Miktar, OrtMaliyet)
VALUES
(1, 'TRY', 'Hisse', 'THYAO', 100, 300.00);
```

-- 13. Müşterilik Başvurusu (Aday Kullanıcı)

```
INSERT INTO MusterilikBasvurulari (KullaniciID, Durum, OnaylayanPersonelID) VALUES
(5, 'Beklemede', NULL); -- Henüz onaylanmadı
```

-- 14. Kampanya Başvurusu

```
INSERT INTO KampanyaBasvurulari (MusteriID, KampanyaID, Durum) VALUES
(2, 1, 'Beklemede'); -- Mehmet "Yeni Müşteri Kredisi"ne başvurdu
```

-- 15. Teknik Destek Talebi

```
INSERT INTO TeknikDestek (MusteriID, IlgiliHesapID, Kanal, Konu, Aciklama, Oncelik) VALUES
(1, 1, 'Uygulama', 'Mobil Giriş Hatası', 'Uygulamaya girerken 404 hatası alıyorum.', 'Orta');
```

## 8. SQL KOMUTLARI VE SCRIPT DOSYALARI

### 8.1 Gereksinim Bazlı SQL Komutları ve Script Dosyaları

**Gereksinim 1:** Müşteri Hesap Özeti ve Güncel Bakiyeler Belirli bir müşterinin (Örn: MüşteriID = 1) sahip olduğu tüm hesapları, para birimlerini ve son işlem sonrası güncel bakiyelerini listeler.

#### SQL Komutu:

```
SELECT
    K.Ad,
    K.Soyad,
    YH.HesapNo,
    YH.ParaBirimiKodu,
    -- Bakiyeyi hesap hareketlerinden çekmek için alt sorgu
    (SELECT TOP 1 HH.IslemSonrasiBakiye
     FROM HesapHareketleri HH
     WHERE HH.HesapID = YH.HesapID
     ORDER BY HH.IslemTarihi DESC) AS GuncelBakiye
FROM YatirimHesaplari YH
INNER JOIN Musteriler M ON YH.MusteriID = M.MusteriID
INNER JOIN Kullanicilar K ON M.KullaniciID = K.KullaniciID
WHERE M.MusteriID = 1; -- Parametre olarak müşteri ID verilir
```

Script Dosyası: queries/musteri\_hesap\_ozeti.sql

**Gereksinim 2:** Son Hesap Hareketleri (Ekstre) Bir hesabın (Örn: HesapID = 5) gerçekleştirdiği son 10 işlemi; işlem türü, tutar ve açıklama bilgileriyle birlikte listeler.

#### SQL Komutu:

```
SELECT TOP 10
    HH.IslemTarihi,
    HH.IslemTuru,
    HH.Tutar,
    HH.ParaBirimiKodu,
    HH.Referans,
    HH.IslemSonrasiBakiye
FROM HesapHareketleri HH
WHERE HH.HesapID = 5 -- Parametre: İlgili Hesap ID
ORDER BY HH.IslemTarihi DESC;
```

Script Dosyası: queries/hesap\_ekstresi.sql

**Gereksinim 3:** Bekleyen Müşteri Başvuruları Henüz onaylanmamış ("Beklemede") olan müşteri başvurularını, başvuran kişinin iletişim bilgileriyle birlikte raporlar.

#### SQL Komutu:

```
SELECT
    MB.BasvuruID,
    K.Ad,
    K.Soyad,
    K.KullaniciNo AS TCKN,
    K.Telefon,
    MB.BasvuruTarihi,
    MB.Durum
FROM MusterilikBasvurulari MB
INNER JOIN Kullanicilar K ON MB.KullaniciID = K.KullaniciID
WHERE MB.Durum = 'Beklemede'
ORDER BY MB.BasvuruTarihi ASC;
```

**Script Dosyası:** queries/bekleyen\_basvurular.sql

**Gereksinim 4:** Banka Genel Varlık Raporu Bankadaki tüm hesaplarda bulunan toplam parayı, para birimi bazında gruplayarak listeler. (Her hesabın son bakiyesini alıp toplar).

#### SQL Komutu:

```
WITH SonBakiyeler AS (
    SELECT
        YH.ParaBirimiKodu,
        -- Her hesap için son işlemdeki bakiyeyi bulur
        (SELECT TOP 1 IslemSonrasiBakiye
         FROM HesapHareketleri HH
         WHERE HH.HesapID = YH.HesapID
         ORDER BY HH.IslemTarihi DESC) AS Bakiye
    FROM YatirimHesaplari YH
    WHERE YH.AktifMi = 1
)
SELECT
    ParaBirimiKodu,
    SUM(ISNULL(Bakiye, 0)) AS ToplamMevduat
FROM SonBakiyeler
GROUP BY ParaBirimiKodu;
```

**Script Dosyası:** queries/genel\_varlik\_raporu.sql

**Gereksinim 5:** Süresi Dolmak Üzere Olan Kredi Kartları Son kullanma tarihi yaklaşan (Örn: Belirli bir yıl/ay filtresi) kartları ve müşteri bilgilerini listeler.

#### SQL Komutu:

```
SELECT
    K.Ad,
    K.Soyad,
    BK.KartNo,
    BK.KartTuru,
    BK.SonKullanma -- Format: MM/YY
FROM BankaKartlari BK
INNER JOIN Musteriler M ON BK.MusteriID = M.MusteriID
INNER JOIN Kullanıcılar K ON M.KullaniciID = K.KullaniciID
WHERE BK.SonKullanma = '12/25'; -- Parametre: MM/YY formatında aranacak tarih
```

**Script Dosyası:** queries/kart\_suresi\_dolumu.sql

**Gereksinim 6:** Açık Teknik Destek Talepleri Çözülmemiş ("Acik" veya "Cozumde") olan teknik destek taleplerini, talebin önceliğine göre sıralayarak listeler.

#### SQL Komutu:

```
SELECT
    TD.TalepID,
    TD.Konu,
    TD.Oncelik,
    TD.Durum,
    TD.AcilisTarihi,
    K.Ad + ' ' + K.Soyad AS MusteriAdSoyad
FROM TeknikDestek TD
INNER JOIN Musteriler M ON TD.MusteriID = M.MusteriID
INNER JOIN Kullanıcılar K ON M.KullaniciID = K.KullaniciID
WHERE TD.Durum IN ('Acik', 'Cozumde')
ORDER BY
    CASE TD.Oncelik
        WHEN 'Yuksek' THEN 1
        WHEN 'Orta' THEN 2
        WHEN 'Dusuk' THEN 3
    END;
```

**Script Dosyası:** queries/aktif\_destek\_talepleri.sql

**Gereksinim 7:** Yüksek Hacimli Transfer İşlemleri Belirli bir tutarın (Örn: 50.000) üzerindeki para transferlerini gönderen ve alan hesap bilgileriyle listeler.



## SQL Komutu:

```
SELECT
    T.TransferID,
    T.Tutar,
    T.ParaBirimiKodu,
    T.IslemTarihi,
    KGonderen.Ad + ' ' + KGonderen.Soyad AS Gonderen,
    KAlıcı.Ad + ' ' + KAlıcı.Soyad AS Alıcı
FROM Transferler T
-- Gönderen Bilgisi
INNER JOIN YatirimHesaplari YHG ON T.GonderenHesapID = YHG.HesapID
INNER JOIN Musteriler MG ON YHG.MusteriID = MG.MusteriID
INNER JOIN Kullanicilar KGonderen ON MG.KullaniciID = KGonderen.KullaniciID
-- Alıcı Bilgisi
INNER JOIN YatirimHesaplari YHA ON T.AliciHesapID = YHA.HesapID
INNER JOIN Musteriler MA ON YHA.MusteriID = MA.MusteriID
INNER JOIN Kullanicilar KAlıcı ON MA.KullaniciID = KAlıcı.KullaniciID
WHERE T.Tutar >= 50000;
```

**Script Dosyası:** queries/yuksek\_hacimli\_transferler.sql

## 9. SAKLI YORDAM VE TETİKLEYİCİ

### 9.1 Saklı Yordam: sp\_DovizIslemi

#### 9.1.1.a Amaç ve İş Operasyonu

Bu saklı yordam, müşterilerin farklı para birimlerindeki hesapları arasında döviz alım ve satım işlemlerini yönetir. Kritik iş operasyonu, anlık kur bilgisini dinamik olarak seçmek (Alış veya Satış kuru), kaynak hesaptaki bakiyeyi kontrol etmek ve atomik bir işlem (Transaction) içinde parayı bir hesaptan düşüp diğerine çevrilmiş tutar olarak eklemektir.

#### 9.1.2.a Saklı Yordam Kodu

```
CREATE PROCEDURE sp_DovizIslemi
    @MusteriID INT,
    @KaynakHesapID INT, -- TL Hesabı
    @HedefHesapID INT, -- USD/EUR Hesabı
    @KaynakTutar DECIMAL(18,2), -- Ne kadar TL bozdurulacak?
    @IslemTuru NVARCHAR(10) -- 'Alis' veya 'Satis'
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION; -- İşlem bütünlüğü başlangıcı

        -- 1. Hedef Para Birimi ve Kur Bilgisini Al
        DECLARE @Kur DECIMAL(18,4);
        DECLARE @HedefParaBirimi CHAR(3);
        SELECT @HedefParaBirimi = ParaBirimiKodu FROM YatirimHesaplari WHERE HesapID = @HedefHesapID;

        -- İşlem türüne göre kuru seç
        IF @IslemTuru = 'Alis'
            SELECT @Kur = SatisKuru FROM ParaBirimleri WHERE ParaBirimiKodu = @HedefParaBirimi;
        ELSE
            SELECT @Kur = AlisKuru FROM ParaBirimleri WHERE ParaBirimiKodu = @HedefParaBirimi;

        -- 2. Hedef Tutarı Hesapla
        DECLARE @HedefTutar DECIMAL(18,2);
        IF @IslemTuru = 'Alis'
            SET @HedefTutar = @KaynakTutar / @Kur;
        ELSE
            SET @HedefTutar = @KaynakTutar * @Kur;

        -- 3. Bakiye Kontrolü
        DECLARE @KaynakBakiye DECIMAL(18,2);
        SELECT TOP 1 @KaynakBakiye = IslemSonrasiBakiye FROM HesapHareketleri WHERE HesapID =
        @KaynakHesapID ORDER BY IslemTarihi DESC;

        IF ISNULL(@KaynakBakiye, 0) < @KaynakTutar
        BEGIN
            RAISERROR('Yetersiz Bakiye!', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        -- 4. Kaynak Hesaptan Düş
        INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, IslemSonrasiBakiye, Referans)
```

```

VALUES (@KaynakHesapID, 'TRY', @IslemTuru, -@KaynakTutar, (@KaynakBakiye - @KaynakTutar),
'Döviz İşlemi');

-- 5. Hedef Hesaba Ekle
DECLARE @HedefBakiye DECIMAL(18,2);
SELECT TOP 1 @HedefBakiye = IslemSonrasiBakiye FROM HesapHareketleri WHERE HesapID =
@HedefHesapID ORDER BY IslemTarihi DESC;

INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, IslemSonrasiBakiye, Referans)
VALUES (@HedefHesapID, @HedefParaBirimi, @IslemTuru, @HedefTutar, (ISNULL(@HedefBakiye,0) +
@HedefTutar), 'Döviz İşlemi');

COMMIT TRANSACTION;
PRINT 'Döviz işlemi başarıyla tamamlandı.';
END TRY
BEGIN CATCH
ROLLBACK TRANSACTION;
DECLARE @HataMesaji NVARCHAR(MAX) = ERROR_MESSAGE();
RAISERROR(@HataMesaji, 16, 1);
END CATCH
END;

```

### 9.1.3.a Test Senaryoları ve Sonuçları

**Senaryo:** Müşteri 32.850 TL verip USD almak istiyor (Kur: 32.85).

**Girdi:** @KaynakTutar = 32850, @IslemTuru = 'Alis', Kaynak: TL Hesabı, Hedef: USD Hesabı.

**Beklenen Sonuç:** TL hesabından 32.850 düşülür, USD hesabına 1000 USD eklenir. HesapHareketleri tablosunda iki yeni kayıt oluşur.

**Hata Senaryosu:** Hesabında 500 TL olan müşteri 1000 TL'lik döviz almak isterse sistem "Yetersiz Bakiye!" hatası fırlatır ve işlemi iptal eder.

### 9.1.b Saklı Yordam: sp\_KartBorcOdeme

#### 9.1.1.b Amaç ve İş Operasyonu

Bu yordam, müşterinin vadesiz hesabından kredi kartı borcunu ödemesini sağlar. Operasyon, hesap bakiyesini kontrol eder ve ödemeyi hesaptan düşerek kart hareketlerine yansıtır. İşlem bütünlüğü **TRANSACTION** bloğu ile sağlanır; bakiye yetersizse süreç geri alınır (**ROLLBACK**).

#### 9.1.2.b Saklı Yordam Kodu

```

CREATE PROCEDURE sp_KartBorcOdeme
    @HesapID INT, -- Vadesiz Hesap
    @KartID INT, -- Ödenecek Kart
    @Tutar DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY

```

```

BEGIN TRANSACTION;
-- 1. Vadesiz Hesap Bakiye Kontrolü
DECLARE @HesapBakiye DECIMAL(18,2);
SELECT TOP 1 @HesapBakiye = IslemSonrasiBakiye FROM HesapHareketleri WHERE HesapID =
@HesapID ORDER BY IslemTarihi DESC;

IF ISNULL(@HesapBakiye, 0) < @Tutar
BEGIN
    RAISERROR('Hesap bakiyesi borç ödemesi için yetersiz!', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END

-- 2. Hesaptan Para Düş
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, IslemSonrasiBakiye, Referans)
VALUES (@HesapID, 'TRY', 'Cek', -@Tutar, (@HesapBakiye - @Tutar), 'KK Borç Ödeme');

-- (Kart bakiyesi güncelleme adımları burada uygulanır)

COMMIT TRANSACTION;
PRINT 'Kredi kartı borcu ödendi.';
END TRY
BEGIN CATCH
    ROLLBACK TRANSACTION;
    RAISERROR(ERROR_MESSAGE, 16, 1);
END CATCH
END;

```

### 9.1.3.b Test Senaryoları ve Sonuçları

**Senaryo:** Müşteri vadesiz hesabından 5.000 TL kredi kartı borcu öder.

**Girdi:** @HesapID = 101, @KartID = 55, @Tutar = 5000.

**Beklenen Sonuç:** HesapHareketleri tablosuna işlem türü 'Cek' olan -5.000 TL'lik kayıt eklenir. Vadesiz hesap bakiyesi 5.000 TL azalır.

**Hata Senaryosu:** Hesabında sadece 1.000 TL bulunan müşteri 5.000 TL borç ödemeye çalışırsa sistem "Hesap bakiyesi yetersiz!" hatası fırlatır ve işlemi iptal eder.

### 9.1.c Saklı Yordam: sp\_YatirimAlimi

#### 9.1.1.c Amaç ve İş Operasyonu

Müşterilerin borsa veya fon gibi yatırım araçlarını satın almasını sağlar. Operasyon, önce nakit bakiyeyi kontrol eder, parayı hesaptan düşer ve ardından HesapVarliklari tablosunda ilgili enstrümanı günceller (varsa üzerine ekler, yoksa yeni kayıt oluşturur/Upsert mantığı).

#### 9.1.2.c Saklı Yordam Kodu

```

CREATE PROCEDURE sp_YatirimAlimi
    @HesapID INT,
    @EnstrumanKodu NVARCHAR(20), -- Örn: THYAO
    @Adet INT,
    @BirimFiyat DECIMAL(18,2)

```

```

AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        DECLARE @ToplamTutar DECIMAL(18,2) = @Adet * @BirimFiyat;

        -- 1. Nakit Bakiye Kontrolü ve Hesaptan Düşüş
        DECLARE @NakitBakiye DECIMAL(18,2);
        SELECT TOP 1 @NakitBakiye = IslemSonrasiBakiye FROM HesapHareketleri WHERE HesapID = @HesapID
        ORDER BY IslemTarihi DESC;

        IF ISNULL(@NakitBakiye, 0) < @ToplamTutar
        BEGIN
            RAISERROR('Yatırım için yetersiz bakiye!', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, IslemSonrasiBakiye, Referans)
        VALUES (@HesapID, 'TRY', 'Alis', -@ToplamTutar, (@NakitBakiye - @ToplamTutar), 'Yatırım: ' +
        @EnstrumanKodu);

        -- 2. Varlık Tablosuna Ekle (Upsert)
        IF EXISTS (SELECT 1 FROM HesapVarliklari WHERE HesapID = @HesapID AND EnstrumanKodu =
        @EnstrumanKodu)
        BEGIN
            UPDATE HesapVarliklari SET Miktar = Miktar + @Adet WHERE HesapID = @HesapID AND
            EnstrumanKodu = @EnstrumanKodu;
        END
        ELSE
        BEGIN
            INSERT INTO HesapVarliklari (HesapID, ParaBirimiKodu, EnstrumanTuru, EnstrumanKodu, Miktar,
            OrtMaliyet)
            VALUES (@HesapID, 'TRY', 'Hisse', @EnstrumanKodu, @Adet, @BirimFiyat);
        END

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
        DECLARE @Msg NVARCHAR(MAX) = ERROR_MESSAGE();
        RAISERROR(@Msg, 16, 1);
    END CATCH
END;

```

### 9.1.3.c Test Senaryoları ve Sonuçları

**Senaryo:** Müşteri tanesi 300 TL'den 10 adet THYAO hissesi satın alır.

**Girdi:** @HesapID = 202, @EnstrumanKodu = 'THYAO', @Adet = 10, @BirimFiyat = 300.

**Beklenen Sonuç:** Nakit bakiyeden 3.000 TL düşülür. HesapVarliklari tablosunda THYAO hissesi varsa miktarı 10 artırılır, yoksa yeni kayıt oluşturulur.

**Hata Senaryosu:** 2.000 TL bakiyesi olan müşteri 3.000 TL'lik hisse alımı yapmaya çalışırsa sistem "Yatırım için yetersiz bakiye!" uyarısı verir ve işlemi durdurur.

### 9.1.d Saklı Yordam: sp\_ParaTransferi

#### 9.1.1.d Amaç ve İş Operasyonu

İki hesap arasında para transferi (EFT/Havale) işlemini yönetir. En kritik iş kuralı, gönderen ve alıcı hesapların para birimlerinin aynı olması zorunluluğudur. Ayrıca işlem bütünlüğünü sağlayarak paranın bir taraftan çıkıp diğer tarafa girmesini garanti eder.

#### 9.1.2.d Saklı Yordam Kodu

```
CREATE PROCEDURE sp_ParaTransferi
    @GonderenHesapID INT,
    @AliciHesapID INT,
    @Tutar DECIMAL(18,2),
    @Aciklama NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        -- 1. Bakiye Kontrolü
        DECLARE @MevcutBakiye DECIMAL(18,2);
        SELECT TOP 1 @MevcutBakiye = IslemSonrasiBakiye FROM HesapHareketleri WHERE HesapID =
        @GonderenHesapID ORDER BY IslemTarihi DESC;

        IF ISNULL(@MevcutBakiye, 0) < @Tutar
        BEGIN
            RAISERROR('Yetersiz Bakiye!', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION;
    END CATCH
END
```

#### 9.1.3.d Test Senaryoları ve Sonuçları

**Senaryo:** Müşteri A hesabından B hesabına 1.000 TL havale yapar.

**Girdi:** @GonderenHesapID = 10, @AliciHesapID = 20, @Tutar = 1000.

**Beklenen Sonuç:** Gönderen hesaptan 1.000 TL düşülür, alıcı hesaba 1.000 TL eklenir. Transferler tablosuna işlem detayı kaydedilir.

**Hata Senaryosu:** TL hesabından USD hesabına doğrudan transfer emri verilirse sistem "Farklı para birimleri arasında doğrudan transfer yapılamaz!" hatası verir.

### 9.1.e Saklı Yordam: sp\_HesapKapat

#### 9.1.1.e Amaç ve İş Operasyonu

Müşterinin kapatmak istediği bir hesabı işleme alır. Güvenlik gereği veritabanından veri silinmez, sadece `AktifMi` kolonu pasife çekilir. Ancak hesapta hala bakiye varsa, paranın kaybolmaması için kapatma işlemi engellenir.

#### 9.1.2.e Saklı Yordam Kodu

```
CREATE PROCEDURE sp_HesapKapat
    @HesapID INT
AS
BEGIN
    -- 1. Bakiye Kontrolü
    DECLARE @SonBakiye DECIMAL(18,2);
    SELECT TOP 1 @SonBakiye = IslemSonrasiBakiye
    FROM HesapHareketleri WHERE HesapID = @HesapID ORDER BY IslemTarihi DESC;

    IF @SonBakiye > 0
    BEGIN
        RAISERROR('Bakiyesi bulunan hesap kapatılamaz! Lütfen önce parayı çekin.', 16, 1);
        RETURN;
    END

    -- 2. Hesabı Pasife Çek
    UPDATE YatirimHesaplari
    SET AktifMi = 0
    WHERE HesapID = @HesapID;

    PRINT 'Hesap başarıyla kapatıldı.';
END;
```

#### 9.1.3.e Test Senaryoları ve Sonuçları

**Senaryo:** Müşteri bakiyesi 0 olan vadesiz hesabını kapatmak ister.

**Girdi:** @HesapID = 88.

**Beklenen Sonuç:** YatirimHesaplari tablosunda ilgili hesabın 'AktifMi' değeri 0 (Pasif) yapılır. Veri fiziksel olarak silinmez.

**Hata Senaryosu:** İçinde 50 TL bakiye unutulmuş bir hesap kapatılmak istenirse sistem "Bakiyesi bulunan hesap kapatılamaz!" uyarısı verir ve işlemi engeller.

## 9.2 Tetikleyici (Trigger)

### 9.2.a trg\_KampanyaTarihKontrol

### 9.2.1.a Amaç ve Tetikleme Koşulu

Kampanya tanımlamalarında yapılabilecek mantıksal hataları önler. Kampanyalar tablosuna veri girildiğinde veya güncellendiğinde (AFTER INSERT, UPDATE) çalışır. Bitiş tarihinin başlangıç tarihinden önce olup olmadığını kontrol eder.

### 9.2.2.a Tetikleyici Kodu

```
CREATE TRIGGER trg_KampanyaTarihKontrol
ON Kampanyalar
AFTER INSERT, UPDATE
AS
BEGIN
    IF EXISTS (SELECT * FROM inserted WHERE BitisTarihi IS NOT NULL AND BitisTarihi < BaslangicTarihi)
    BEGIN
        RAISERROR('Hata: Kampanya bitiş tarihi, başlangıç tarihinden önce olamaz!', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```

### 9.2.3.a Test Senaryoları ve Sonuçları

**Senaryo:** Personel sisteme hatalı tarihli bir kampanya girmeye çalışır.

**Girdi:** INSERT Komutu (BaslangicTarihi: '2024-05-01', BitisTarihi: '2024-04-01').

**Beklenen Sonuç:** YatirimHesaplari tablosunda ilgili hesabın 'AktifMi' değeri 0 (Pasif) yapılır. Veri fiziksel olarak silinmez.

## 9.2.b trg\_ParaBirimiTarihGuncelle

### 9.2.1.b Amaç ve Tetikleme Koşulu

Döviz kurları sürekli değiştiğinden, kullanıcının gördüğü kurun ne zaman güncellendiğini bilmesi gerekir. Bu trigger, ParaBirimleri tablosundaki kurlar güncellendiğinde (UPDATE) otomatik olarak GuncellemeTarihi sütununa o anki zamanı yazar.

### 9.2.2.b Tetikleyici Kodu

```
CREATE TRIGGER trg_ParaBirimiTarihGuncelle
ON ParaBirimleri
AFTER UPDATE
AS
BEGIN
    IF UPDATE(AlisKuru) OR UPDATE(SatisKuru)
    BEGIN
        UPDATE ParaBirimleri
        SET GuncellemeTarihi = GETDATE()
        FROM ParaBirimleri P
        INNER JOIN inserted I ON P.ParaBirimiKodu = I.ParaBirimiKodu;
    END
END;
```

### 9.2.3.b Test Senaryoları ve Sonuçları



**Senaryo:** Piyasalarda USD kuru değişir ve sistemde güncelleme yapılır.

**Girdi:** UPDATE ParaBirimleri SET Satiskuru = 32.60 WHERE ParaBirimiKodu = 'USD'.

**Beklenen Sonuç:** YatirimHesaplari tablosunda ilgili hesabın 'AktifMi' değeri 0 (Pasif) yapılır. Veri fiziksel olarak silinmez.

**Hata Senaryosu:** GuncellemeTarihi sütunu, sorguda belirtilmemesine rağmen trigger tarafından otomatik olarak o anki sistem saati ile değiştirilir.

## 9.2.c trg\_HareketDegisikligiEngelle

### 9.2.1.c Amaç ve Tetikleme Koşulu

Bankacılık sisteminin en temel güvenlik kuralı olan "Değiştirilemezlik (Immutability)" ilkesini uygular. HesapHareketleri tablosundan bir kayıt silinmeye veya değiştirilmeye çalışıldığında devreye girer (INSTEAD OF DELETE, UPDATE) ve işlemi reddeder.

### 9.2.2.c Tetikleyici Kodu

```
CREATE TRIGGER trg_HareketDegisiklikEngelle
ON HesapHareketleri
INSTEAD OF DELETE, UPDATE
AS
BEGIN
    RAISERROR('Güvenlik Uyarısı: Geçmiş hesap hareketleri silinemez veya güncellenemez! Lütfen ters işlem (iade) yapınız.', 16, 1);
END;
```

### 9.2.3.c Test Senaryoları ve Sonuçları

**Senaryo:** Bir kullanıcı geçmiş tarihli bir hesap hareketini silmeye çalışır.

**Girdi:** DELETE FROM HesapHareketleri WHERE HareketID = 105.

**Beklenen Sonuç(Hata):** Trigger işlemi durdurur, kayıt silinmez ve kullanıcıya "Güvenlik Uyarısı: Geçmiş hesap hareketleri silinemez!" mesajı gösterilir.

## 9.2.d trg\_BasvuruDurumLog

### 9.2.1.d Amaç ve Tetikleme Koşulu

Müşteri başvurularının durum değişikliklerini (Örn: Beklemede -> Onaylandı) takip etmek ve denetim (audit) izi oluşturmak için kullanılır. MusterilikBasvurulari tablosunda durum değiştiğinde eski ve yeni durumu loglar.

### 9.2.2.d Tetikleyici Kodu

```
CREATE TRIGGER trg_BasvuruDurumLog
ON MusterilikBasvurulari
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Durum)
    BEGIN
        DECLARE @EskiDurum NVARCHAR(20);
        DECLARE @YeniDurum NVARCHAR(20);
        DECLARE @BasvuruID INT;

        SELECT @EskiDurum = Durum FROM deleted;
        SELECT @YeniDurum = Durum, @BasvuruID = BasvuruID FROM inserted;

        INSERT INTO SistemLoglari (TabloAdi, IslemTuru, Aciklama)
        VALUES ('MusterilikBasvurulari', 'UPDATE', 'BasvuruID: ' + CAST(@BasvuruID AS NVARCHAR) + '
durumu değişti. Eski: ' + @EskiDurum + ' -> Yeni: ' + @YeniDurum);
    END
END;
```

### 9.2.3.c Test Senaryoları ve Sonuçları

**Senaryo:** Bir kredi başvurusunun durumu personel tarafından güncellenir.

**Girdi:** UPDATE MusterilikBasvurulari SET Durum = 'Onaylandı' WHERE BasvuruID = 45.

**Beklenen Sonuç:** SistemLoglari tablosuna otomatik olarak "BasvuruID: 45 durumu değişti. Eski: Beklemede -> Yeni: Onaylandı" şeklinde bir denetim kaydı eklenir.

## 9.2.e trg\_HesapSilmeKorumasi

### 9.2.1.e Amaç ve Tetikleme Koşulu

Banka hesaplarının kazara silinmesini önler. YatirimHesaplari tablosunda silme işlemi yapıldığında (INSTEAD OF DELETE) devreye girer ve kullanıcıyı silme yerine "Pasife Çekme" (Soft Delete) yöntemine yönlendirir.

### 9.2.2.e Tetikleyici Kodu

```
CREATE TRIGGER trg_HesapSilmeKorumasi
ON YatirimHesaplari
INSTEAD OF DELETE
AS
BEGIN
    RAISERROR('Güvenlik Uyarısı: Banka hesapları fiziksel olarak silinemez. Lütfen hesabı pasife (AktifMi=0)
çekiniz.', 16, 1);
```

END;

### 9.2.3.c Test Senaryoları ve Sonuçları

**Senaryo:** Veritabanı yöneticisi yanlışlıkla bir hesap kaydını tablodan silmeye çalışır.

**Girdi:** DELETE FROM YatirimHesaplari WHERE HesapID = 12.

**Beklenen Sonuç:** Silme işlemi engellenir ve sistem "Banka hesapları fiziksel olarak silinemez. Lütfen hesabı pasife çekiniz." uyarısını döner.

## 10. TRANSACTION YÖNETİMİ

### 10.1 Döviz Alım Satım Senaryosu

Müşterinin yerel para birimi (TRY) hesabından, döviz (USD/EUR) hesabına para aktarması işlemidir. Bu işlem, anlık kurun çekilmesini, kaynak hesaptan TL düşülmesini ve hedef hesaba kur karşılığı döviz eklenmesini kapsar. sp\_DovizIslemi içinde bu adımlar tek bir transaction bloğunda yönetilir.

### 10.2 Transaction Kodu

```
CREATE PROCEDURE sp_DovizIslemi
    @MusteriID INT,
    @KaynakHesapID INT,
    @HedefHesapID INT,
    @KaynakTutar DECIMAL(18,2),
    @IslemTuru NVARCHAR(10)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION; -- Atomik işlem başlangıcı

        -- Kur Çekme ve Hesaplama Adımları... (Kısaltılmıştır)
        -- ...

        -- Bakiye Yetersizse ROLLBACK
        IF ISNULL(@KaynakBakiye, 0) < @KaynakTutar
        BEGIN
            RAISERROR('Yetersiz Bakiye!', 16, 1);
            ROLLBACK TRANSACTION;
            RETURN;
        END

        -- Kaynak Hesaptan Düş (INSERT INTO HesapHareketleri...)
        -- Hedef Hesaba Ekle (INSERT INTO HesapHareketleri...)

        COMMIT TRANSACTION; -- Başarılı
        PRINT 'Döviz işlemi başarıyla tamamlandı.';
    END TRY
    BEGIN CATCH
        ROLLBACK TRANSACTION; -- Hata durumunda iptal
        DECLARE @HataMesaji NVARCHAR(MAX) = ERROR_MESSAGE();
        RAISERROR(@HataMesaji, 16, 1);
    END CATCH
END;
```

### 10.3 Başarılı Senaryo Testi

Test Kodu:

```
-- Hazırlık: Hesap 1 (TRY) bakiyesi 49.000 TL. Hesap 2 (USD) bakiyesi 0.
-- Kur: 32.85 (Varsayım)
-- İşlem: 32.850 TL karşılığında Dolar Al.
```

```
EXEC sp_DovizIslemi
  @MusteriID = 1,
  @KaynakHesapID = 1,
  @HedefHesapID = 2,
  @KaynakTutar = 32850.00,
  @IslemTuru = 'Alis';
```

Beklenen **Sonuç**:

**Mesaj**: Döviz işlemi başarıyla tamamlandı.

Hesap 1 (TRY): -32.850 TL azalır.

Hesap 2 (USD): +1.000 USD artar.

**COMMIT** işlemi onaylar.

## 10.4 Hata Senaryosu ve ROLLBACK Testi

-- Hazırlık: Hesap 1'de yeterli bakiye yokmuş gibi davranılır (Örn: Hesapta 5 TL var).

-- İşlem: 100.000 TL'lik döviz almaya çalış.

```
EXEC sp_DovizIslemi
  @MusteriID = 1,
  @KaynakHesapID = 1, -- (İçi boş varsayılan hesap)
  @HedefHesapID = 2,
  @KaynakTutar = 100000.00,
  @IslemTuru = 'Alis';
```

Beklenen **Sonuç**:

Hata **Mesajı**: Msg 50000, Level 16, State 1: Yetersiz Bakiye!

Hiçbir hesaptan para düşülmez.

**ROLLBACK** devreye girerek işlemi geri alır.

## 10.2 Kredi Kartı Borç Ödeme Senaryosu

Bu senaryo, müşterinin vadesiz hesabından (nakit) kredi kartı borcunu ödemesi işlemi yönetir. İşlem atomiktir; yani vadesiz hesap bakiyesi kontrol edilir, yeterliyse para hesaptan düşülür ve işlem onaylanır. Eğer bakiye yetersizse veya sistemsel bir hata oluşursa işlem ROLLBACK ile geri alınır, böylece hesaptan para düşüp borcun ödenmemiş görünmesi gibi durumlar engellenir.

### 10.2 Transaction Kodu

```
CREATE PROCEDURE sp_KartBorcOdeme
  @HesapID INT, -- Vadesiz Hesap
  @KartID INT, -- Ödenecek Kart
  @Tutar DECIMAL(18,2)
AS
BEGIN
  SET NOCOUNT ON;
  BEGIN TRY
    -- Transaction Başlatma
    BEGIN TRANSACTION;

    -- 1. Vadesiz Hesap Bakiye Kontrolü
```

```

DECLARE @HesapBakiye DECIMAL(18,2);
SELECT TOP 1 @HesapBakiye = IslemSonrasiBakiye
FROM HesapHareketleri
WHERE HesapID = @HesapID
ORDER BY IslemTarihi DESC;

-- Bakiye Yetersizse İptal Et (ROLLBACK)
IF ISNULL(@HesapBakiye, 0) < @Tutar
BEGIN
    RAISERROR('Hesap bakiyesi borç ödemesi için yetersiz!', 16, 1);
    ROLLBACK TRANSACTION;
    RETURN;
END

-- 2. Hesaptan Para Düş (Eksi Bakiye Hareketi)
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, IslemSonrasiBakiye, Referans)
VALUES (@HesapID, 'TRY', 'Cek', -@Tutar, (@HesapBakiye - @Tutar), 'KK Borç Ödeme');

-- 3. İşlem Başarılıysa Onayla
COMMIT TRANSACTION;
PRINT 'Kredi kartı borcu ödendi.';

END TRY
BEGIN CATCH
    -- Hata durumunda işlemi geri al
    ROLLBACK TRANSACTION;
    RAISERROR(ERROR_MESSAGE, 16, 1);
END CATCH
END;

```

## 10.2 Başarılı Senaryo Testi

Bu senaryoda, bakiyesi borç tutarını karşılayan bir hesaptan ödeme yapılır.

```

-- Hazırlık: Hesap 1'de 5.000 TL bakiye var.
-- İşlem: 1.200 TL kart borcu öde.
EXEC sp_KartBorcOdeme
    @HesapID = 1,
    @KartID = 101, -- Müşterinin kart ID'si
    @Tutar = 1200.00;

```

Beklenen **Sonuç**:

**Mesaj**: Kredi kartı borcu ödendi.

Hesap 1 **Bakiyesi**: 3.800 TL'ye düşer.

HesapHareketleri tablosuna "KK Borç Ödeme" referanslı çıkış kaydı eklenir.

Veritabanında COMMIT çalışmıştır

## 10.4 Hata Senaryosu ve ROLLBACK Testi

Bu senaryoda, bakiyesi yetersiz olan bir hesaptan ödeme yapılmaya çalışılır.

```

-- Hazırlık: Hesap 1'de 500 TL bakiye kaldı.
-- İşlem: 2.000 TL borç ödemeye çalış.
EXEC sp_KartBorcOdeme
    @HesapID = 1,
    @KartID = 101,
    @Tutar = 2000.00;

```

Beklenen **Sonuç**:

Hata Mesajı: Msg 50000, Level 16, State 1: Hesap bakiyesi borç ödemesi için yetersiz!

Hesap 1 Bakiyesi: Değişmez (500 TL olarak kalır).

Veritabanında ROLLBACK çalışır, para düşülmez.

### 10.3 Hesaplar Arası Para Transferi Senaryosu

Bu senaryo, bir göndericinin hesabından para düşülmesi ve alıcının hesabına aynı miktarın eklenmesi işlemidir. Bu iki adım atomik olmalıdır; yani gönderenden para düşülüp alıcıya eklenemezse (sistem hatası, elektrik kesintisi vb.), işlem tamamen iptal edilmeli ve gönderenin parası iade edilmelidir (ROLLBACK). Ayrıca, işlem sırasında farklı para birimleri arasında doğrudan transfer yapılması engellenerek veri tutarlılığı sağlanmalıdır.

### 10.3 Transaction Kodu

```
CREATE PROCEDURE sp_ParaTransferi
    @GonderenHesapID INT,
    @AliciHesapID INT,
    @Tutar DECIMAL(18,2),
    @Aciklama NVARCHAR(100)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        -- Transaction Başlatma: Bu noktadan sonraki işlemler sanaldır.
        BEGIN TRANSACTION;

        -- 1. Bakiye Kontrolü
        DECLARE @MevcutBakiye DECIMAL(18,2);
        SELECT TOP 1 @MevcutBakiye = IslemSonrasiBakiye
        FROM HesapHareketleri
        WHERE HesapID = @GonderenHesapID
        ORDER BY IslemTarihi DESC;

        IF ISNULL(@MevcutBakiye, 0) < @Tutar
        BEGIN
            RAISERROR('Yetersiz Bakiye! İşlem gerçekleştirilemedi.', 16, 1);
            ROLLBACK TRANSACTION; -- Hata varsa geri al
            RETURN;
        END

        -- 2. Para Birimi Kontrolü (Business Logic)
        DECLARE @GonderenParaBirimi CHAR(3), @AliciParaBirimi CHAR(3);
        SELECT @GonderenParaBirimi = ParaBirimiKodu FROM YatirimHesaplari WHERE HesapID =
        @GonderenHesapID;
        SELECT @AliciParaBirimi = ParaBirimiKodu FROM YatirimHesaplari WHERE HesapID = @AliciHesapID;

        IF @GonderenParaBirimi <> @AliciParaBirimi
        BEGIN
            RAISERROR('Hata: Farklı para birimleri arasında doğrudan transfer yapılamaz!', 16, 1);
            ROLLBACK TRANSACTION; -- Hata varsa geri al
            RETURN;
        END

        -- 3. Transfer Kaydı (Master)
```

```

DECLARE @YeniTransferID INT;
INSERT INTO Transferler (GonderenHesapID, AliciHesapID, Tutar, ParaBirimiKodu, Aciklama)
VALUES (@GonderenHesapID, @AliciHesapID, @Tutar, @GonderenParaBirimi, @Aciklama);
SET @YeniTransferID = SCOPE_IDENTITY();

-- 4. Gönderenden Düş (Detail 1)
INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, TransferID, IslemTuru, Tutar,
IslemSonrasiBakiye, Referans)
VALUES (@GonderenHesapID, @GonderenParaBirimi, @YeniTransferID, 'TransferGiden', -@Tutar,
(@MevcutBakiye - @Tutar), @Aciklama);

-- 5. Alıcıya Ekle (Detail 2)
DECLARE @AliciBakiye DECIMAL(18,2);
SELECT TOP 1 @AliciBakiye = IslemSonrasiBakiye FROM HesapHareketleri WHERE HesapID =
@AliciHesapID ORDER BY IslemTarihi DESC;

INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, TransferID, IslemTuru, Tutar,
IslemSonrasiBakiye, Referans)
VALUES (@AliciHesapID, @AliciParaBirimi, @YeniTransferID, 'TransferGelen', @Tutar,
(ISNULL(@AliciBakiye,0) + @Tutar), @Aciklama);

-- İşlem Başarılı: Kalıcı Hale Getir
COMMIT TRANSACTION;
PRINT 'Transfer başarıyla gerçekleşti.';

END TRY
BEGIN CATCH
-- Beklenmedik bir hata olursa her şeyi geri al
ROLLBACK TRANSACTION;
DECLARE @HataMesaji NVARCHAR(MAX) = ERROR_MESSAGE();
RAISERROR(@HataMesaji, 16, 1);
END CATCH
END;

```

### 10.3 Başarılı Senaryo Testi

Bu senaryoda, bakiyesi yeterli olan bir hesaptan aynı para birimine sahip başka bir hesaba transfer yapılır.

-- Hazırlık: Hesap 1'de 50.000 TL var, Hesap 3'te 0 TL var.

-- İşlem: 1000 TL gönder.

EXEC sp\_ParaTransferi

@GonderenHesapID = 1,

@AliciHesapID = 3,

@Tutar = 1000.00,

@Aciklama = 'Borç Ödeme';

Beklenen Sonuç:

Mesaj: Transfer başarıyla gerçekleşti.

Hesap 1 Bakiyesi: 49.000 TL

Hesap 3 Bakiyesi: 1.000 TL

Veritabanında COMMIT çalışmıştır.

### 10.3 Hata Senaryosu ve ROLLBACK Testi

Bu senaryoda, bakiyesi yetersiz olan bir hesaptan para gönderilmeye çalışılır.

-- Hazırlık: Hesap 1'de 49.000 TL kaldı.



-- İşlem: 1.000.000 TL göndermeye çalış (Yetersiz Bakiye).

EXEC sp\_ParaTransferi

@GonderenHesapID = 1,

@AliciHesapID = 3,

@Tutar = 1000000.00,

@Aciklama = 'Araba Alımı';

Beklenen Sonuç:

Hata Mesajı: Msg 50000, Level 16, State 1: Yetersiz Bakiye! İşlem gerçekleştirilemedi.

Hesap 1 Bakiyesi: Değişmez (49.000 TL)

Hesap 3 Bakiyesi: Değişmez (1.000 TL)

Veritabanında ROLLBACK çalışmış, Transferler tablosuna boş kayıt atılmamıştır.

## 10.4 Yatırım Varlığı Alımı Senaryosu

Bu senaryo, müşterinin vadesiz hesabındaki nakit parayı kullanarak hisse senedi, fon veya tahvil gibi bir yatırım aracı satın almasını yönetir. İşlem atomiktir; önce nakit bakiye kontrol edilir ve düşülür, ardından "Upsert" (Update veya Insert) mantığı ile varlık tablosu güncellenir. Eğer müşteri o varlığa zaten sahipse miktar artırılır, sahip değilse yeni bir varlık kaydı oluşturulur. Herhangi bir adımda hata olursa (yetersiz bakiye vb.) tüm işlemler ROLLBACK ile geri alınır.

## 10.4 Transaction Kodu

CREATE PROCEDURE sp\_YatirimAlimi

@HesapID INT,

@EnstrumanKodu NVARCHAR(20), -- Örn: THYAO

@Adet INT,

@BirimFiyat DECIMAL(18,2) -- O anki borsa fiyatı

AS

BEGIN

SET NOCOUNT ON;

BEGIN TRY

-- Transaction Başlatma

BEGIN TRANSACTION;

DECLARE @ToplamTutar DECIMAL(18,2) = @Adet \* @BirimFiyat;

-- 1. Nakit Bakiye Kontrolü

DECLARE @NakitBakiye DECIMAL(18,2);

SELECT TOP 1 @NakitBakiye = IslemSonrasiBakiye

FROM HesapHareketleri

WHERE HesapID = @HesapID

ORDER BY IslemTarihi DESC;

-- Bakiye Yetersizse İptal (ROLLBACK)

IF ISNULL(@NakitBakiye, 0) < @ToplamTutar

BEGIN

RAISERROR('Yatırım için yetersiz bakiye!', 16, 1);

ROLLBACK TRANSACTION;

RETURN;

END

-- 2. Nakit Hesaptan Düş (Para Çıkışı)

INSERT INTO HesapHareketleri (HesapID, ParaBirimiKodu, IslemTuru, Tutar, IslemSonrasiBakiye, Referans)

```

VALUES (@HesapID, 'TRY', 'Alis', -@ToplamTutar, (@NakitBakiye - @ToplamTutar), 'Yatırım: ' +
@EnstrumanKodu);

-- 3. Varlık Tablosuna Ekle (Yoksa oluştur, varsa güncelle)
IF EXISTS (SELECT 1 FROM HesapVarliklari WHERE HesapID = @HesapID AND EnstrumanKodu =
@EnstrumanKodu)
BEGIN
    -- Varlık zaten portföyde varsa miktarını artır
    UPDATE HesapVarliklari
    SET Miktar = Miktar + @Adet
    WHERE HesapID = @HesapID AND EnstrumanKodu = @EnstrumanKodu;
END
ELSE
BEGIN
    -- Varlık portföyde yoksa yeni kayıt aç
    INSERT INTO HesapVarliklari (HesapID, ParaBirimiKodu, EnstrumanTuru, EnstrumanKodu, Miktar,
OrtMaliyet)
    VALUES (@HesapID, 'TRY', 'Hisse', @EnstrumanKodu, @Adet, @BirimFiyat);
END

-- İşlemi Onayla
COMMIT TRANSACTION;
PRINT 'Yatırım işlemi gerçekleşti.';

END TRY
BEGIN CATCH
    -- Hata durumunda işlemi geri al
    ROLLBACK TRANSACTION;
    DECLARE @Msg NVARCHAR(MAX) = ERROR_MESSAGE();
    RAISERROR(@Msg, 16, 1);
END CATCH
END;

```

### 10.3 Başarılı Senaryo Testi

Bu senaryoda, bakiyesi yeterli olan bir müşteri portföyünde olmayan yeni bir hisse senedi alır.

```

-- Hazırlık: Hesap 1'de 10.000 TL bakiye var.
-- İşlem: Tanesi 50 TL'den 100 adet ASELS hissesi al (Toplam 5.000 TL).
EXEC sp_YatirimAlimi
    @HesapID = 1,
    @EnstrumanKodu = 'ASELS',
    @Adet = 100,
    @BirimFiyat = 50.00;

```

Beklenen **Sonuç**:

**Mesaj:** Yatırım işlemi gerçekleşti.

Hesap 1 **Bakiyesi:** 5.000 TL azalır.

HesapVarliklari tablosuna 'ASELS' koduyla 100 adetlik yeni bir satır eklenir.

Veritabanında **COMMIT** çalışmıştır.

### 10.3 Hata Senaryosu ve ROLLBACK Testi

Bu senaryoda, müşteri bakiyesinden daha fazla tutarda hisse senedi almaya çalışır.

```

-- Hazırlık: Hesap 1'de 5.000 TL kaldı.
-- İşlem: Tanesi 1.000 TL'den 10 adet hisse almaya çalış (Toplam 10.000 TL - Yetersiz).

```

EXEC sp\_YatirimAlimi  
@HesapID = 1,  
@EnstrumanKodu = 'THYAO',  
@Adet = 10,  
@BirimFiyat = 1000.00;  
Beklenen Sonuç:

Hata Mesajı: Msg 50000, Level 16, State 1: Yatırım için yetersiz bakiye!

Hesap 1 Bakiyesi: Değişmez (5.000 TL).

HesapVarliklari tablosunda herhangi bir değişiklik olmaz.

Veritabanında ROLLBACK çalışır.

## 11. TAKIM ÇALIŞMASI VE GÖREV DAĞILIMI

### 11.1 Takım Üyeleri ve Roller

| Takım Üyesi | Öğrenci No | Temel Rol                                      |
|-------------|------------|------------------------------------------------|
| Mustafa     |            | Proje Lideri / Veri Tabanı Mimarı              |
| Şeyhmus     |            | SQL Geliştirici                                |
| Emirhan     |            | SQL Geliştirici / Test ve Dokümantasyon Uzmanı |

### 11.2 Gerçekleştirilen İşler ve Sorumluluk Matrisi

| No | Yapılan İş / Özellik                                | Talimat Veren | Gerçekleştiren | Kontrol Eden | Durum |
|----|-----------------------------------------------------|---------------|----------------|--------------|-------|
| 1  | Proje gereksinimlerinin belirlenmesi                | Mustafa       | Tüm Ekip       | Tüm Ekip     | ✓     |
| 2  | E-R diyagramının oluşturulması                      | Mustafa       | Mustafa        | Tüm Ekip     | ✓     |
| 3  | İlişkisel şemalara dönüştürme                       | Mustafa       | Mustafa        | Tüm Ekip     | ✓     |
| 4  | Normalizasyon (3NF/BCNF)                            | Mustafa       | Tüm Ekip       | Şeyhmus      | ✓     |
| 5  | SQL Server’da şema oluşturma (Create Tables)        | Mustafa       | Şeyhmus        | Emirhan      | ✓     |
| 6  | Birincil/Yabancı anahtar tanımları                  | Şeyhmus       | Emirhan        | Emirhan      | ✓     |
| 7  | Nitelik kısıtlamalarının eklenmesi (Check/Default)  | Şeyhmus       | Emirhan        | Şeyhmus      | ✓     |
| 8  | Örnek verilerin eklenmesi (Insert Scripts)          | Mustafa       | Emirhan        | Emirhan      | ✓     |
| 9  | SQL sorgu script dosyalarının hazırlanması (Select) | Mustafa       | Şeyhmus        | Emirhan      | ✓     |
| 10 | Saklı yordamın (Stored Procedure) yazılması         | Mustafa       | Şeyhmus        | Mustafa      | ✓     |
| 11 | Tetikleyicinin (Trigger) yazılması                  | Mustafa       | Şeyhmus        | Emirhan      | ✓     |
| 12 | Transaction yönetimi kodunun yazılması              | Mustafa       | Şeyhmus        | Mustafa      | ✓     |
| 13 | COMMIT/ROLLBACK test senaryoları                    | Şeyhmus       | Emirhan        | Şeyhmus      | ✓     |
| 14 | Rapor hazırlama ve dokümantasyon                    | Mustafa       | Emirhan        | Mustafa      | ✓     |
| 15 | Son kontrol ve teslim                               | Mustafa       | Tüm Ekip       | Tüm Ekip     | ✓     |

## 12. SONUÇ VE DEĞERLENDİRME

Bu proje kapsamında, modern bankacılık standartlarına uygun, veri güvenliğini, işlem bütünlüğünü ve izlenebilirliği merkeze alan kapsamlı bir **Banka Otomasyon Sistemi (BOS)** veritabanı mimarisi tasarlanmış ve MS SQL Server üzerinde başarıyla uygulanmıştır. Proje süreci, teorik veritabanı tasarımı prensiplerinin (Normalizasyon, E-R Modelleme) pratik uygulamalarla (T-SQL, Stored Procedures, Triggers) birleştirilerek çalışan, güvenli bir sisteme dönüştürülmesini sağlamıştır.

### 12.1 Elde Edilen Deneyimler ve Kazanımlar

Proje geliştirme sürecinde, veritabanı yönetim sistemleri üzerine aşağıdaki teknik yetkinlikler ve deneyimler kazanılmıştır:

- **İleri Seviye T-SQL Uygulamaları:** Karmaşık iş mantıklarının uygulama katmanı yerine veritabanı katmanında çözülmesi hedeflenmiş; bu amaçla Saklı Yordamlar ve Tetikleyiciler etkin bir şekilde kullanılmıştır.
- **Transaction Yönetimi:** Finansal sistemlerin temel taşı olan **ACID** (Atomicity, Consistency, Isolation, Durability) prensipleri sistemin merkezine yerleştirilmiştir. Özellikle para transferi ve döviz işlemleri gibi kritik süreçlerde, olası hata durumlarında veri tutarlılığını korumak için **COMMIT** ve **ROLLBACK** mekanizmalarının önemi uygulamalı olarak deneyimlenmiştir.
- **Veri Güvenliği ve Denetim (Audit):** Güvenliğin sadece şifreleme ile sınırlı olmadığı; verinin tarihçesinin korunmasının da kritik olduğu anlaşılmıştır. **INSTEAD OF** triggerları ile geçmiş kayıtların silinmesi engellenmiş ve kritik veri değişikliklerinin loglanması sağlanarak "Denetim İzi" (Audit Trail) oluşturulmuştur.
- **Veritabanı Normalizasyonu:** Veri tekrarını önlemek, disk alanını verimli kullanmak ve güncelleme anormalliklerinin önüne geçmek amacıyla tüm tablolar **BCNF (Boyce-Codd Normal Form)** standartlarına uygun hale getirilmiştir.

### 12.2 Karşılaşılan Zorluklar ve Çözümler

Projenin tasarım ve kodlama aşamalarında karşılaşılan temel teknik zorluklar ve bu zorluklara üretilen mimari çözümler şunlardır:

1. **Zorluk:** Hesaplar arası para transferi sırasında, gönderici hesaptan paranın düşülmesi ancak alıcı hesaba eklenememesi (sistemsel kesinti vb.) riski.
  - **Çözüm:** Transfer işlemi tek bir **Transaction** bloğu içerisine alınarak "Ya Hep Ya Hiç" prensibi uygulanmıştır. İşlem adımlarının tamamı başarılı olmadan veritabanına kaydedilmemesi, herhangi bir adımda hata alınması durumunda ise tüm işlemlerin geri alınması (Rollback) sağlanmıştır.
2. **Zorluk:** Yatırım işlemlerinde (Hisse/Fon alımı), müşterinin portföyünde ilgili varlığın olup olmamasına göre farklı işlem (Ekleme veya Güncelleme) yapılması gerekliliği.
  - **Çözüm:** Kod karmaşasını önlemek için "Upsert" mantığı kurgulanmıştır. **IF EXISTS** kontrolü ile varlık mevcutsa miktar güncellenmiş, mevcut değilse yeni kayıt oluşturulmuş; böylece veri tutarlılığı korunmuştur.

3. **Zorluk:** Geçmişe dönük finansal verilerin manipüle edilmesinin (silinmesi veya değiştirilmesi) önlenmesi.
  - **Çözüm:** Kritik tablolara (Örn: HesapHareketleri), DELETE ve UPDATE işlemlerini engelleyen ve kullanıcıya güvenlik uyarısı döndüren özel tetikleyiciler yazılarak veritabanı seviyesinde "Değiştirilemezlik" (Immutability) ilkesi hayata geçirilmiştir.

### 12.3 Genel Değerlendirme

Sonuç olarak; oluşturulan bu proje ile gerçek bir bankacılık sisteminin çekirdek (Core Banking) fonksiyonlarını karşılayan, ölçeklenebilir ve güvenli bir altyapı ortaya konulmuştur. Tasarlanan sistem, sadece verilerin saklandığı bir depo değil; iş kurallarını denetleyen, hataları yöneten ve veri bütünlüğünü aktif olarak koruyan "akıllı" bir yapı olarak kurgulanmıştır. Bu çalışma, büyük ölçekli ve kritik veri barındıran sistemlerin mimari tasarımı konusunda önemli bir yetkinlik kazandırmıştır.